



UPPSALA
UNIVERSITET

UPTEC IT 25023

Degree project 30 credits

June, 2025

Comparative Analysis of Models for Document Question Answering using Retrieval-Augmented Generation

Markus Lindgren
Lukas Rådmann

Master's Programme in Computer and Information
Engineering



UPPSALA
UNIVERSITET

Comparative Analysis of Models for Document Question Answering using Retrieval-Augmented Generation

Markus Lindgren
Lukas Rådman

Abstract

This thesis presents a comparative analysis of Large Language Models (LLMs) for Swedish document question answering using Retrieval-Augmented Generation (RAG). The goal has been to evaluate how well the Nordic model GPT-SW3 compares to state-of-the-art models Llama 3.2 and DeepSeek-V3. Additionally, research has been conducted on how the performance of the question-answering system changes when the RAG pipeline is enhanced. These topics have been addressed by implementing three different RAG pipelines with varying complexity, based on Swedish data related to environmental certification of buildings. These pipelines were then evaluated in combination with three LLMs through automatically calculating metrics with LLM-as-a-judge and through human evaluation by domain experts. The results show that GPT-SW3 does not match the performance of Llama 3.2 and DeepSeek-V3 on this task, with DeepSeek-V3 achieving the best scores. The findings also indicate that the performance of the question-answering system can be increased by refining the RAG pipeline. However, the performance gap between the pipelines diminishes when leveraging a more sophisticated model to generate the answers. This research offers practical insights into selecting LLMs and designing RAG systems, especially for Swedish, highlighting challenges for smaller models against global competitors.

Faculty of Science and Technology
Uppsala University, Uppsala

Supervisor: Viktor Mattsson Subject reader: Alexander Medvedev
Examiner: Roland Bol

Sammanfattning

Denna studie jämför tre stora språkmodeller, GPT-SW3, Llama 3.2 och DeepSeek-V3, i fråga-svar-system för svenska texter med hjälp av Retrieval-Augmented Generation (RAG). RAG är en teknik som gör att en språkmodell kan använda sig av information från ett specifikt ämnesområde. Det innebär att modellen kan ge mer relevanta och pålitliga svar, eftersom den hämtar kunskap från en databas med fakta. Studien fokuserar på hur väl dessa modeller klarar att svara på frågor baserade på svenska texter, i detta fall om miljöcertifiering av byggnader. GPT-SW3 är en nordisk modell utvecklad för det svenska språket, medan Llama 3.2 och DeepSeek-V3 är internationella modeller. Tre RAG-system med olika teknisk komplexitet utvecklades och testades i kombination med varje modell. Svaren utvärderades både automatiskt, med hjälp av AI, och manuellt av ämnesexperter. Resultaten visar att GPT-SW3 presterar avsevärt sämre än de internationella modellerna och den bedöms inte kunna konkurrera med de andra modellerna. Däremot visar studien att ett mer avancerat RAG-system kan förbättra resultaten, särskilt för svagare modeller som GPT-SW3. För Llama 3.2 och DeepSeek-V3 hade komplexiteten mindre effekt, eftersom de producerade bra resultat oavsett systemets uppbyggnad. Studien ger praktiska insikter kring hur val av språkmodell och systemarkitektur påverkar systemets förmåga att svara på frågor om svenska miljömanualer, och belyser både möjligheter och begränsningar med språkspecifika modeller i jämförelse med globala alternativ.

Acknowledgment We would like to express our sincere gratitude to Softhouse for giving us the opportunity to carry out our master's thesis within their organization. A special thanks goes to the team at the Uppsala office for their warm welcome, support, and valuable input throughout the project. Also worth mentioning is the dartboard, which provided many well-needed breaks and occasional bruised egos. We are especially grateful to our supervisor, Viktor Mattsson, whose guidance, feedback, and encouragement have been crucial to the progress and outcome of our work.

Contents

Acronyms	vii
1 Introduction	1
1.1 Motivation	2
1.2 Research Questions	2
1.3 Thesis Outline	3
2 Background & Theory	4
2.1 Large Language Models	4
2.1.1 GPT-SW3	5
2.1.2 Llama 3.2	5
2.1.3 DeepSeek-V3	5
2.2 Question-Answering Systems	6
2.3 Retrieval-Augmented Generation	7
2.3.1 Document Chunking	7
2.3.2 Embeddings	8
2.3.3 Vector Database	9
2.3.4 Techniques for Improving RAG	10
2.4 Evaluation	11
3 Related Work	13
3.1 Question-Answering with RAG	13
3.1.1 Swedish RAG Applications	13
3.1.2 Comparison of Large Language Models in RAG	14
3.2 Evaluation Methods	14

4	Methodology	16
4.1	Data Collection & Preparation	16
4.2	RAG Implementation	18
4.2.1	Shared Characteristics	18
4.2.2	Naive Pipeline	20
4.2.3	Semi-Structured Pipeline	20
4.2.4	Hybrid Pipeline	21
4.3	Selection of LLMs for GPT-SW3 Comparison	22
4.4	Evaluation	23
4.4.1	Test Dataset	23
4.4.2	Retrieval Metrics	24
4.4.3	Generation Metrics	25
4.4.4	RAGAs Experiments	26
4.4.5	Human Evaluation	28
5	Results	30
5.1	Retrieval Results	30
5.2	Generation Performance Across Pipelines	32
5.3	Performance of Large Language Models	36
5.4	Results From Human Evaluation	40
6	Discussion	41
6.1	Discussion of Results	41
6.1.1	Research Question 2	41
6.1.2	Research Question 1	43
6.2	Discussion of Methods	44

6.3	Broader Impacts and Considerations	46
7	Conclusions	48
A	Appendix	56
B	Contributions	59

Acronyms

LLM Large Language Model. 1–7, 9–14, 16, 22–30, 32, 35–40, 42, 44–48

NLP Natural Language Processing. 1, 4, 6–9, 13, 44, 48

Q&A Question-Answer. 1–4, 6, 13–15, 48

RAG Retrieval-Augmented Generation. 1–4, 6, 7, 9–11, 13–16, 18–20, 23, 24, 28, 30, 32–37, 40, 42–46, 48

1 Introduction

After the release of OpenAI’s chatbot, ChatGPT, in November 2022, Generative Artificial Intelligence has become widely discussed. Since November 2022, the development of generative AI has seen exponential growth, with AIs now capable of creating movie-quality images or solving a variety of coding tasks [71][5]. The main building blocks of text-to-text applications like ChatGPT are Large Language Models (LLMs). These models have shown impressive performance on Natural Language Processing (NLP) tasks including text summarization, generation, and sentiment analysis [57]. The development of these LLMs has turned into a competitive race among large companies striving to create the most advanced models to gain an edge in the evolving field of AI.

With the immense attention garnered by ChatGPT and competing models from other major tech companies, most of which have been trained on predominantly English data, the interest in language-specific models has surfaced. One example is the recent development and release of AI Sweden’s model GPT-SW3 [19]. This model is the first native large generative model for the Nordic Germanic languages, with a particular emphasis on Swedish. Although this model is trained on less data than state-of-the-art models, it shows promising results in generating responses in Swedish [19]. However, LLMs are often discarded when the factual accuracy of the generated text is critical. This is because LLMs are trained on vast amounts of data and rely on pattern recognition rather than fact-based sources, which can lead to text generations that are incorrect or complete fiction, known as hallucinations [70]. Different techniques to mitigate their hallucinations and specialize the models in various topics have emerged. One of these techniques is Retrieval-Augmented Generation (RAG). This technique connects an LLM to an external database to decrease the hallucinations by including factual and specialized content in the prompts [36]. RAG also allows the LLM to access information it has not been trained on, such as company-specific data or newly generated data beyond the cut-off date of its training.

Retrieval-Augmented Generation systems have become a key technology in advancing the accuracy of chatbots [23]. This thesis evaluates the performance of GPT-SW3 in comparison to other LLMs in a RAG system for Question-Answer (Q&A). To conduct this evaluation, three different RAG system with varying complexity were developed, each utilizing external data, specifically manuals related to environmental certification of buildings. An environmental certification of a building is a tool to assess a building’s sustainability and environmental impact. Obtaining such a certification can enhance the building’s market value, reduce operating costs, and demonstrate a commitment to environmental responsibility [14][59]. The certification considered in this work is “Miljöbyggnad”, a tool developed by the Sweden Green Building Council (SGBC), which also serves as one of the external stakeholders for this project.

1.1 Motivation

This thesis is motivated by two factors: first, it contributes to the research on LLMs and RAG for non-English languages, in this case Swedish, and second, it provides a possible practical application in the realm of environmental certification of buildings.

Academic Motivation This thesis explores the growing field of RAG and investigates the performance of various LLMs within a RAG system using Swedish as the primary language. Although similar research has been done before [24][47], we aim to further explore this topic by comparing how the most advanced Nordic language model, GPT-SW3, compares to state-of-the-art models from major tech companies in different RAG setups.

Practical Motivation The stakeholder for this project is Softhouse, a software development and digital transformation consulting firm. They aim to leverage this technique to create a system that can answer company-specific data. For this thesis, the data will be acquired from one of Softhouse's customers in the field of environmental certification of buildings. By creating a RAG model that can be used as a Q&A system, the process of environmental certification for buildings can be streamlined, providing easier access to information and improving overall efficiency in achieving certification standards. Thus, this project aims to provide Softhouse with insights on how different RAG setups and LLMs perform in this particular domain.

1.2 Research Questions

This project aims to address the following research questions, focusing on the evaluation of Large Language Models within the context of a RAG system applied to a specific domain:

- Q1** How does GPT-SW3 perform on a Document Question Answering task within a Retrieval-Augmented Generation system, compared to other state-of-the-art Large Language Models?
- Q2** How does the performance of the question-answering system change when the Retrieval-Augmented Generation system is made more advanced?

1.3 Thesis Outline

This thesis is divided into multiple chapters as outlined below.

Background & Theory This section provides a background of the topics discussed in the paper, starting with Large Language Models. Furthermore, it gives an overview of Question-Answer systems and a technical introduction to a Retrieval-Augmented Generation system and its components.

Related Work In this section, we provide a review of existing literature on Retrieval-Augmented Generation systems, including the use of Large Language Models within RAG pipelines, progress in Swedish language applications of RAG, and evaluation methods.

Methodology This section presents our implementation and the motivation behind technical decisions. We introduce our three RAG pipelines and how we have implemented them. Lastly, we motivate and describe our evaluation experiments based on automated and human evaluations.

Results In this section, we present the results from our automated and human evaluation. Figures and tables containing our results are shown, and we comment briefly on notable results.

Discussion Here we reflect on and discuss the results obtained from our experiments. We discuss and draw several conclusions by connecting the results to our research questions. Limitations and possible improvements of our method and a discussion on broader impacts are also presented in this section.

Conclusion In this section, we summarize our findings related to the research questions and conclude our thesis.

2 Background & Theory

This section provides background theory relevant to this thesis, covering Large Language Models, Question-Answer systems, Retrieval-Augmented Generation, and evaluation techniques.

2.1 Large Language Models

Large Language Models are foundation models in the field of Natural Language Processing. The models are trained to understand and generate natural language content. LLMs are based on the transformer architecture, first presented by Google [61]. Transformers consist of multiple layers of neural networks that are trained on massive amounts of data. The development of LLMs marked an upswing in generative AI, with a breakthrough occurring when OpenAI introduced their GPT-3 model in 2020 [4]. Today, there are a vast number of publicly available models, such as OpenAI GPT models, Google's Gemini, and Meta's Llama.

One key differentiating factor between these models is the amount of information they can process in a single prompt. This is commonly referred to as a model's context window, which is the maximum number of tokens a model can handle at once. For example, OpenAI's GPT-3 has a context window limited to 2,048 tokens [4], while Google's Gemini 1.5 Pro, released in 2024, supports up to 2 million tokens [62]. The expansion of the context window is one example of the rapid advancement of LLMs in recent years. A larger context window allows models to handle longer inputs with more information. However, research has shown that extensive prompts with too much text can cause LLMs to overlook information [39].

Training LLMs involves predicting the next word or segment in a sentence based on the surrounding words. LLMs can learn to find conceptual and semantic relations by self-supervised learning. Self-supervised learning differs from supervised learning in that it does not require existing labels; instead, the model learns to predict parts of the input with the context of the remaining data. The prediction of words is done by calculating a probability score for possible words that could fit, and choosing one of the most probable words. It is important to note that the models do not always choose the most probable next word, since this would lead to deterministic models that always give the same answer to a specific question. This yields models that can generate natural language that is contextually relevant based on the input.

2.1.1 GPT-SW3

GPT-SW3 is a collection of native large generative language models for the Nordic languages developed by AI Sweden, RISE, and WASP WARA Media & Language [19]. The models are decoder language models specifically designed to generate language. The GPT-SW3 models are trained on approximately 320B tokens in Norwegian, Danish, Icelandic, Swedish, and English, and the different model sizes range from 120M up to 40B parameters. The dataset used for training was created from various text sources, such as the Norwegian Colossal Corpus, the Danish and Icelandic Gigaword corpora, data repositories, websites, and discussion forums in Swedish. Additionally, the dataset contains the programming languages Python, JavaScript, SQL, and Shell script. The model collection is accessible through AI Sweden’s Hugging Face repository [1].

2.1.2 Llama 3.2

Meta has developed Llama 3.2, a family of multilingual, pretrained, and instruction-tuned generative models. This collection builds on the foundation model series Llama 3 [26], which uses a standard dense transformer architecture similar to earlier versions, Llama and Llama 2 [61]. Improvements in Llama 3 are achieved by better data quality, more diverse data, and extended training. Llama 3.2 further develops this with an optimized transformer architecture, tuned by reinforcement learning from human feedback and supervised fine-tuning. While Llama 3.2 officially supports English, Italian, Portuguese, Hindi, Spanish, German, French, and Thai, it has also been trained on additional languages, with a total training dataset comprising 9 trillion tokens. The models, available in 1B and 3B parameter sizes, are openly accessible via Meta’s Hugging Face repository [40].

2.1.3 DeepSeek-V3

In December 2024, the Chinese AI company Deepseek released an open-source LLM called Deepseek-V3 [15]. It created a lot of buzz in the world of AI as it outperformed other open- and closed-source models such as OpenAI’s GPT-4o, while claiming a development cost of about \$5M to build [3]. This cost claim has been questioned, and replication of the model is required to verify it. However, this has not yet been done.

Deepseek-V3 is a 617B parameter Mixture-of-Experts language model with 37B activated for each token. The model’s foundation is built using a Transformer architecture [61]. Additionally, to improve the inference efficiency and cost of training, Multi-Head Latent Attention (MLA) and DeepseekMoE are used [15]. MLA reduces the cost of

attention by introducing a small set of latent tokens that serve as intermediaries. Input tokens attend to these latent tokens instead of directly to each other, which makes the attention computation more efficient. This is especially effective for long input sequences. DeepSeekMoE is a Mixture-of-Experts design where only a subset of neural networks is activated per input. This allows the model to have hundreds of billions of parameters without using all of them at once. MoE results in lower computational overhead and higher efficiency while keeping strong performance. Deepseek-V3 is pre-trained with 14.8 trillion tokens and optimized with reinforcement learning from human feedback and supervised fine-tuning.

2.2 Question-Answering Systems

Question-Answer (Q&A) systems is a subfield of Natural Language Processing and focuses on answering natural language questions from a user. There are two main approaches to Q&A systems, open- or closed-domain [6]. The first one refers to a system that is capable of answering any type of question in any domain, whereas the latter only focuses on a closed domain. Early Q&A systems focused on closed-domain architecture with the use of databases populated with a specific topic. Open-domain systems have become more popular, and current research focuses almost exclusively on this topic.

The field of Q&A computer systems stretches back to the 1960s, with the introduction of BASEBALL [27], one of the earliest Q&A systems that could handle natural text input. The system read the user's question from punched cards and parsed the question into a structured query using a rule-based approach with an underlying dictionary. This query was then passed to a database containing baseball statistics such as game scores, dates, and teams. The result from the database query was then printed on paper, demonstrating how basic the computers were back then. Although this system was usable, it lacked scalability and generalizability due to the narrow domain and hand-crafted grammar and database schemas.

Like many other techniques, these systems have become more advanced over the past decades. In 2010, IBM researchers introduced Watson [22], a Q&A system that was able to beat experts in the game of *Jeopardy!*, a famous trivia game show. By training Machine Learning algorithms on vast amounts of data, and utilizing advanced NLP techniques, the system showed the potential of artificial intelligence in the field of Q&A. In the most recent years, Large Language Models have dominated the field with their capabilities of understanding natural language. One of the Q&A techniques leveraging LLMs is Retrieval-Augmented Generation, described in more detail in the following section.

2.3 Retrieval-Augmented Generation

With the rise of Large Language Models in the past few years, document question answering using Retrieval-Augmented Generation has become popular. RAG is a technique for enhancing the answers of an LLM by retrieving information from a database. The retrieved information, along with the user question, is then provided as input to the LLM. The RAG concept for NLP tasks was introduced in 2020 by Facebook and presented a framework with database retrieval functionality combined with a generative language model [36]. Since then, RAG frameworks have been improved, and the technique has shown great results in both minimizing hallucination and providing the LLM with data past its cut-off date. A cut-off date for an LLM is the point in time when the model's training data ends. Any events occurring after this date are not included in the model's training, meaning the LLM may lack awareness of newer information. However, by adding more context, without retraining, the knowledge database can be extended, which makes the LLM aware of more up-to-date information. This makes the approach scalable and highly efficient for applications with a constantly growing database.

Figure 1 presents an overview of a simple RAG application where the user's question is first sent to the retriever. Utilizing embeddings, the retriever fetches relevant data from a vector store based on the similarity between the user's query and the document chunks. Data in the vector database is stored as embedded chunks of the original data instead of complete documents, making it easier to find similarities between the queries and data chunks to retrieve the relevant data. After the relevant data is fetched from the knowledge store, both the user question and the retrieved context are sent to the final generation step, where an LLM answers the question based on the context. In the following subsections, we will dive deeper into each part of the system presented in Figure 1.

2.3.1 Document Chunking

Document chunking plays a crucial role in retrieving the correct information from the knowledge database. Even with the growing context-windows of LLMs, they struggle to identify the relevant information in larger contexts, indicating the need for shorter, more accurate contexts [39]. To address this, documents are split into smaller pieces with semantic meaning that can be effectively indexed and retrieved. The data is stored in discrete chunks, which can be optimized for specific knowledge databases given the characteristics of the original data. Chunking techniques may be based on structures such as sentences, paragraphs, pages, characters, or words, each offering distinct advantages. Large chunks can capture more context but have more noise, compared to small

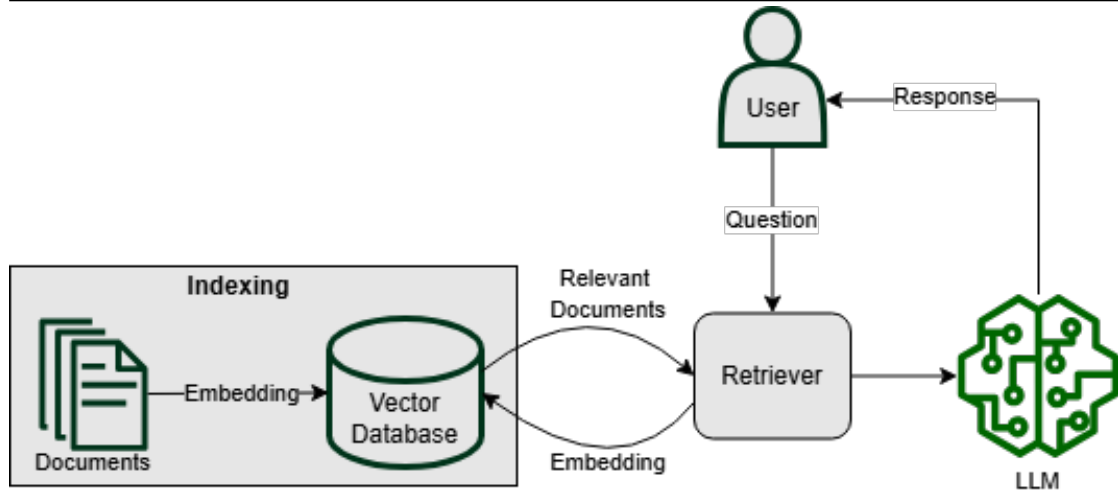


Figure 1: Overview of a RAG application.

chunks, which risk missing context, but have less noise [23]. To address the potential problem of missing context within chunks, a sliding window technique with overlap is often employed. This involves creating overlapping chunks, where each chunk overlaps its adjacent chunks by a specific number of characters, ensuring that crucial context is not lost during retrieval [23]. There are also content-aware chunking techniques based not only on the fixed size of tokens but also on context. These methods include recursive chunking and semantic chunking. Recursive chunking tries to split the data on specific given endpoints such as paragraphs, new lines, and punctuation. If the produced chunks do not meet the desired needs, for example, size or structure, the algorithm recursively continues with the chunking. Semantic or NLP chunking splits data into chunks based on semantics by using advanced NLP tools, such as embeddings, explained in detail in [34]. This way, the text is divided at semantic boundaries, preserving context in the chunks.

2.3.2 Embeddings

To efficiently store and retrieve the chunked data explained in the previous section, the data needs to be embedded [2]. Embeddings are created by either statistical methods or pre-trained models that convert the data chunks into high-dimensional vector representations. These techniques calculate different kinds of vectors that can be broadly categorized into sparse and dense embeddings. Furthermore, the user queries also need to be embedded, allowing the search for relevant segments in the high-dimensional space.

Sparse embeddings are vectors where most of the values are zero. The vectors are gen-

erally very high-dimensional, where each dimension represents a word in a vocabulary. These embeddings are created by leveraging statistical techniques that weigh the importance of each word in a query and assign corresponding values to their respective dimensions in the vector space. Common methods include *Bag-of-Words*, where each vector represents the count of individual words in a query [48], and *TF-IDF* (Term Frequency-Inverse Document Frequency), which extends Bag-of-Words by weighting words based on their frequency within a query relative to their occurrence across an entire document corpus [52]. Sparse embeddings provide great interpretability since each dimension often corresponds to a specific word, making it easy to analyze. However, due to the same reason, these embeddings often fail to capture the semantic relationship between words as they rely on exact matches. When working with large amounts of documents, it can be computationally expensive to store and search these vectors due to their high dimensionality.

The other way of representing texts as vectors is dense embeddings. In contrast to sparse embeddings, dense embeddings have lower dimensionality and the values in the vectors are often non-zero. By leveraging a pre-trained embedding model, vectors can be created where each dimension captures some meaning of the underlying data. These embeddings are more abstract and it can be hard to decipher what each dimension represents, making them less interpretable than sparse embeddings [54]. However, since dense embeddings capture the underlying meaning of words or sentences, they generalize well over different texts. For example, the vectors created by embedding the words “Happy” and “Joyful” would be close together in the vector space since the meanings of the words are similar, whilst sparse embeddings of these words would not generalize as well. One technique to calculate these embeddings is *word2vec*, introduced by Google in 2013 [41]. This method learns word embeddings by training a neural network to predict a target word based on its surrounding context words. With the advancements in Natural Language Processing and LLMs, sophisticated embedding models that can embed entire sentences or documents have been developed. An example of this is the *BGE-M3* model [11]. This model has been trained on a vast amount of text from different languages and can embed documents up to 8,192 tokens long. Additionally, this model can create both dense and sparse embeddings, showcasing its versatility.

The choice of embedding technique influences an application’s retrieval performance and different methods are better suited for various domains and end goals.

2.3.3 Vector Database

Retrieval-Augmented Generation applications leverage a vector database to store the embeddings described in the previous section. In contrast to relational databases, which

are optimized for structured data and logic-based search, a vector database specializes in storing and querying the embedded text vectors. The vectors are embedded in a high-dimensional space, where semantically similar vectors are positioned close to each other. To enable semantic search, a similarity score can be computed between vectors [29]. This score is typically calculated using mathematical functions such as cosine similarity or inner product, depending on the embedding type and the distance metric in the vector store.

As stated in the previous section, sparse embeddings are high-dimensional and capture the exact words in a text, while dense embeddings are lower-dimensional and represent the underlying meaning and semantic relationships. As a result, different documents could be retrieved when searching both types of embeddings with the same query, depending on the words used. One way of combining the strengths of dense and sparse embeddings is to conduct a hybrid search. To do this, both the dense and sparse embeddings of the document need to be calculated and stored in the vector database. Hybrid retrieval then searches both the sparse and dense vector spaces and retrieves the top matches in both categories. Finally, the results are combined, and the similarity score of each document sorts the retrieved documents.

2.3.4 Techniques for Improving RAG

While Retrieval-Augmented Generation already provides a powerful framework for combining information retrieval with text generation, it can be further advanced using specific techniques. Enhancing Retrieval-Augmented Generation involves refinement of user queries, improving the ranking of retrieved documents, and optimizing prompts. This section covers key techniques: question optimization, re-ranking, and prompt formulation to improve retrieval accuracy and LLM responses.

To enhance the retrieval process, we can optimize the users' initial questions, which is called question optimization [23]. User questions are often ambiguous, containing acronyms that can have multiple meanings, which can affect the performance of an embedding model. To mitigate this issue, the user question is rephrased or expanded to provide more context and clarity, ensuring that the embedding model captures the intended meaning accurately.

Documents retrieved from the vector database are initially ranked based on similarity search. However, a re-ranking can provide a more accurate ranking of the contexts. The re-ranking can be done by rule-based methods or different sorts of LLMs. LLMs are influenced by the positioning of information in a prompt and tend to focus more on the edges of the prompt [39]. A re-ranker takes a query and a document as input and returns their relevance score. Unlike embedding models, which encode the query and

document separately, re-ranker models use cross-encoders to process them jointly. By passing both the query and document simultaneously through a transformer network, a more precise relevance prediction can be achieved, allowing for improved re-ranking of retrieval results [43].

Prompt formulation is similar to prompt engineering, where prompts are optimized to get a relevant response from the LLM in the generation step. Prompt compression can be utilized if the retrieved information is too large for the LLM and attempts to compress the information without losing any vital context. Role-prompting is another useful technique in prompt engineering that assigns the LLM a specific persona. By instructing the model to act as an expert in a specific field or as a helpful assistant, its responses can become more aligned with the desired output. This approach can also help shape the style and structure of the response [10].

2.4 Evaluation

Evaluating a RAG application is complex and challenging to define, as it has multiple interacting components. As described in Section 2.1, the output from the LLM in the system is not deterministic, making it difficult to evaluate performance consistently. Further, since RAG systems are often applied to domain-specific tasks, expert knowledge in that area is essential for accurately evaluating their performance. A key part of this evaluation involves judging the relevance of the retrieved contexts [72]. It requires a deep understanding of whether the given context is applicable to answer the question. Human experts are usually good at this task because they can spot subtle connections, understand hidden meanings, and make sense of the text as a whole. Yet, having experts evaluate a RAG system is expensive and does not scale well, especially when dealing with a large knowledge base.

To address scalability, automated evaluation methods often rely on statistical metrics to estimate the relevance of retrieved content by comparing it to a reference or ground truth answer. Common examples include BLEU [46] and ROUGE [37], which heavily rely on measuring overlap in wording and phrasing between the retrieved text and the reference. While these statistical methods are cheap and scalable, they can fail to capture the deeper meaning of the text since they do not account for semantic similarity and cannot assess whether the text is truly relevant [28].

With the rise of LLMs, a new approach to evaluating natural language has emerged. They can be used as evaluators for more complicated and flexible assessments, making more accurate and relevant decisions than previous methods. This has led to the use of LLMs in various evaluation fields referred to as *LLM-as-a-judge*. LLMs can mimic

human experts and, in some domains, achieve similar results much faster and more cost-effectively [28]. However, the main challenge with using LLMs for evaluation is their reliability. The issue of bias and reliability also concerns human evaluators as they are found to have an inherent bias [28]. LLMs have been found to have multiple biases, including information positioning, length, and concreteness bias, which influence how responses are structured and how detailed or complex they are. Furthermore, studies have shown that GPT-4 tends to prefer its own answers to those generated by GPT-3.5. It remains unclear whether this is due to the greater detail in GPT-4’s responses or self-bias [28]. Therefore, it is preferable to use a different LLM for the evaluation than the one used for generation.

A framework that supports statistical metrics and also leverages the strengths of LLM-as-a-judge is RAGAs (Retrieval Augmented Generation Assessment) [21]. The RAGAs framework incorporates several metrics such as Faithfulness, Answer Relevance, and Context Relevance. Most metrics in RAGAs leverage LLM-as-a-judge to calculate the scores, although some can also be calculated using traditional methods such as embedding similarity. The framework can also be used to create comprehensive test datasets, this feature is especially valuable in domains where labeled datasets are nonexistent. RAGAs is developed in Python and can be imported as an external library in projects where extensive evaluation of LLMs is needed.

3 Related Work

This section surveys existing literature on Retrieval-Augmented Generation, covering key architectural advancements, evaluation methods, and applications. We examine foundational RAG research, recent progress in Swedish language applications, and studies comparing Large Language Model performance within RAG pipelines.

3.1 Question-Answering with RAG

The field of Question-Answer systems has had significant growth since Facebook introduced their first RAG system in 2020 [36]. This study introduces a RAG solution that combines parametric pre-trained models with non-parametric memory for generation. By integrating information retrieval using a vector database and a LLM, this study presents a benchmark for early RAG applications. The evaluation of the application was conducted across a wide range of NLP tasks and three Question-Answer datasets.

Since the release of Facebook’s paper, many Retrieval-Augmented Generation solutions for Q&A have been developed for various domains. Frameworks for Q&A in medical, financial, and cybersecurity applications have been published, which proves the wide use of RAG[18][55][49]. More general RAG systems have been developed, such as GARAG [66]. The paper, GARAG: A General adaptive question-answering system based on RAG, incorporates advanced techniques into the traditional RAG pipeline to enhance the quality of generated answers. Like a classical RAG application, the system has a retrieval agent, a generator agent, and a vector database. However, in this solution, the agents use different techniques depending on the user’s question and the retrieved documents. The retrieval process can also be dynamically restarted through continuous evaluation if the retrieved context is irrelevant to the question. The study presents results computed across multiple Q&A datasets, demonstrating that the GARAG solution improves the performance in Q&A tasks [66].

3.1.1 Swedish RAG Applications

As interest in Retrieval-Augmented Generation systems grows, researchers have increasingly investigated their application in non-English contexts, including with Swedish datasets [13] [8]. Adapting LLMs in Swedish RAG systems presents unique challenges, such as limited high-quality training data and domain-specific terminology [30]. Several studies have explored the application of RAG systems with Swedish data, focusing on domain-specific retrieval and generation tasks. For instance, a recent Master’s the-

sis from Kungliga Tekniska Högskolan presents a Q&A system for Swedish technical documentation, showcasing good performance [8]. Another study demonstrates the potential of a Swedish RAG pipeline that can answer IT support tickets [7]. The pipeline evaluation involved a two-week test period during which seven technicians assessed the usefulness of system-generated responses in a live environment. The dataset comprised 401 IT support tickets, of which 154 (38.4%) were resolved without requiring additional manual input. While a 38.4% success rate may appear modest, it demonstrates the potential of Swedish RAG systems and highlights opportunities for further refinement and optimization.

3.1.2 Comparison of Large Language Models in RAG

The effectiveness of Large Language Models in Retrieval-Augmented Generation systems has been a growing area of research, particularly as these models continue to evolve and improve. Evaluating their performance in different RAG setups is crucial for understanding their strengths and limitations across various applications. A recent Master's thesis by students at Dalarna University explores the performance of GPT-3.5 and BERT in a RAG system with Swedish data from Trafikverket [24]. A more comprehensive and up-to-date comparison of larger and more advanced models was presented in [47]. This paper examined GPT-3.5-turbo by OpenAI, Gemini-Pro by Google, and LLama3 by Meta. Although it is a more up-to-date comparison of LLMs, the usefulness of the results can be debated since they use a different embedding model for each LLM. Both studies concluded that OpenAI's GPT models outperformed the alternatives. However, these analyses did not explore whether different RAG configurations influenced the models' performance.

3.2 Evaluation Methods

Approaches to evaluating a RAG application differ, but it is common to evaluate the retriever, the generator, and the system end-to-end [68]. Each of these components presents unique evaluation challenges. Many early evaluation frameworks for RAG systems focused on specific metrics such as retrieval accuracy or factual correctness in the generation step [33]. To calculate retrieval and generation scores, it is common to utilize well-known Q&A datasets such as SQuAD, TriviaQA, and Natural Questions [50] [32] [35]. Google's Natural Questions dataset pairs real user questions from Google with annotated answers in documents, making it a suitable dataset for training and testing RAG and LLM-based Question-Answer systems [35].

More recent studies have proposed frameworks for a complete evaluation of RAG sys-

tems [21][12][53]. An example of such a framework is RAGAs [21], which have been developed to evaluate RAG systems without having access to human-annotated datasets or reference answers. A recent study presents a RAG pipeline for cloud security, using documentation from Amazon Web Services (AWS) as its knowledge base [65]. The system is designed for Q&A with the goal of reducing security vulnerabilities resulting from knowledge gaps. Using RAGAs, the authors generate a dataset of question-answer pairs and calculate several evaluation metrics.

Another study presents a survey of evaluation frameworks for RAG systems [68]. The authors introduce an approach called Auepora (A Unified Evaluation Process of RAG), which classifies existing evaluation benchmarks that focus on evaluation targets, datasets, and metrics. The survey highlights that different frameworks apply to specific parts of the RAG pipeline. It concludes that the choice of evaluation framework should depend on the application domain and objectives of the RAG system.

4 Methodology

In this section, we present and motivate a series of methods and tools to collect and prepare data, implement Retrieval-Augmented Generation pipelines, select Large Language Models and assess system performance through automated and human evaluation. The chosen methods were selected according to the specific requirements and constraints of the task.

4.1 Data Collection & Preparation

The quality of the data in a RAG system is of utmost importance when finding relevant documents in the vector database and providing relevant context to the LLM. The data used in this research consists of publicly available manuals on environmental building certifications from the Sweden Green Building Council (SGBC) [58]. These manuals are guidelines for achieving the “Miljöbyggnad” certification and are available in multiple versions. However, since the differences in these versions are minimal, we chose to use the most up-to-date manual, “Manual Miljöbyggnad Ombyggnad 4.0”. This manual is a 65-page, semi-structured PDF containing a mix of text, tables, and images. Page 17 of the manual is visible in Figure 2 to illustrate the structure of the PDF.

The first step in preprocessing the data was extracting the text and tables from the PDF file into a format compatible with the RAG system. This process presented a challenge in the potential corruption of tables when splitting the PDF, which is a well-known difficulty in the field of information extraction [69]. As illustrated in Figure 3, the structure of certain tables in the PDF was difficult to interpret, even for humans. In this case, “Redovisning” and “Preliminär Certifiering” are on the same row, making it unclear to which of these the bullet points in the rightmost column refer. Other tables in the PDF contained the same information multiple times, or colors and icons were used instead of text. To address this challenge, different techniques were tested to find a suitable solution for extracting the data. Unstructured is an open-source library that provides tools to preprocess documents for LLM usage [60]. The results from the Unstructured implementation were poor, particularly for table extraction, where incorrectly formatted JSON objects would require substantial manual effort to perfect. Another possible solution was LlamaParse, a Generative AI-powered document parser developed by LlamaIndex [38]. Documents can be uploaded and processed directly via the LlamaParse web dashboard or using the API. With many different features and possibilities to tune the parser by preference, it is a powerful tool. The results from LlamaParse were promising but had similar issues with the table extraction as Unstructured. Lastly, we explored the use of image recognition with an LLM. This process involved converting each page

INDIKATOR 2 SOLVÄRMELAST



Syfte

Syftet är att premiera byggnader som projekteras och byggs för att begränsa övertemperaturer och effektbehovet för komfortkyla under sommarhalvåret, samt säkerställa god dagsljustransmittans.

Vad bedöms

Solvärmelasttalet i W/m² golvarea under sommarhalvåret.

Globala mål och Sveriges miljömål

Globala mål		Sveriges miljömål
	God hälsa och välbefinnande	 Hållbara städer och samhällen
		 God bebyggd miljö

Betygskriterier

	BRONS 	SILVER 	GULD 
Bostäder	≤ 38	- Betyg Brons uppfylls. ≤ 29. - LT-värde ≥ 0,6.	- Betyg Silver uppfylls. ≤ 18.
Lokaler	≤ 40	- Betyg Brons uppfylls. ≤ 32. - LT-värde ≥ 0,6.	- Betyg Silver uppfylls. ≤ 22.

Instruktion

Med solvärmelast (SVL) avses den solvärme som passerar fönster och bidrar till uppvärmning av rummet. Solvärmelasttalet definieras som den solvärme som tillförs rummet per kvadratmeter golvarea.

Fasader som vetter mellan 90 och 270°, det vill säga öster till väster via söder, ingår i bedömningen. Rörlig solavskärmning antas vara aktiverad vid beräkning av g_{system} .

I lokaler baseras beräkningar på solavskärmningar som är installerade och i funktion när byggnaden tas i drift.

I bostäder baseras beräkningar på utvändiga och mellanliggande solavskärmningar som är installerad och i funktion vid idrifttagandedatum. I bostäder erfordras ej att invändiga solskydd är monterade när byggnaden tas i drift. Boende ska ha information om vilken typ av invändigt solskydd som uppfyller kriterierna. LT-värdet för samtliga fönster i vistelserum behöver överstiga 0,6. Om detta ej uppfylls behöver godkänd dagsljusberäkning enligt BBR redovisas för det aktuella rummet.

Figure 2: Page 17 from “Manual Miljöbyggnad Ombyggnad 4.0”, demonstrating the structure of the data before preprocessing.

Taxonomikriterium A1-7.1-DNSH 6 		
Kriterium		<i>Taxonomikriterium A1-7.1-DNSH 6</i> Utredning av behovet av miljökonsekvensbedömning.
Lokaler och bostäder		• En miljökonsekvensbedömning eller en prövning av behovet av bedömning har utförts i enlighet med direktiv 2011/92/EU. • Om en miljökonsekvensbedömning har utförts innebär detta att de erforderade riskbegränsnings- och kompensationsåtgärderna för att skydda miljön genomförs.
Redovisning	Preliminär certifiering	• Dokument eller mötesprotokoll som klarlägger behov av miljökonsekvensbedömning. • Eventuell miljökonsekvensbedömning. • Eventuell handling med utpekade åtgärder för att möta identifierade risker i eventuell miljökonsekvensbedömning.

Figure 3: Table from Miljöbyggnad 4.0 demonstrating the complex table structure where rows and columns intertwine.

of the PDF to PNG format, prompting OpenAI’s GPT-4o model with the images, and then concatenating the results into a Markdown file, with tables formatted in JSON. In general, this can be an expensive solution due to the costly API, but since the PDF used was small, it was possible and turned out to be the most accurate solution. The file with the concatenated results demanded some manual correction, mostly fixing the structure of a few JSON table objects. The advantage of using Markdown is its ability to preserve the document’s structural elements, such as headers and subheaders, from the original PDF.

4.2 RAG Implementation

Based on Research Question 2 (Q2) in Section 1.2, three RAG pipelines with increasing complexity were developed. First, shared characteristics of the three pipelines are introduced, and then specific differences in each of the implemented pipelines are presented. The three pipelines are henceforth referred to as the naive pipeline, semi-structured pipeline, and hybrid pipeline.

4.2.1 Shared Characteristics

The three pipelines shared several features, starting with the development framework, *LangChain* [9]. This library is a common tool for RAG pipelines and enables seamless integration between the system components.

All three systems utilized a vector database for storing the embedded text extracted

from the PDF. A local instance of the *Milvus* vector database was set up using Docker [63] [16]. Milvus was chosen based on its advanced search capabilities and seamless integration with our RAG pipeline. Milvus can store both sparse and dense vector embeddings for the same document and supports hybrid search on these. The decision to use Milvus was based on a study of different vector databases, and we also considered using *Pinecone* [67] and *FAISS* [17]. Pinecone is a vector database as a service. It is a fully managed, scalable alternative that supports real-time vector search through its API. These features are very attractive for fast deployment of, for example, RAG applications where scaling is needed. However, due to the scope of the project and cost constraints, we chose to disregard Pinecone as an option. FAISS is a vector database developed by Meta and is optimized for local development on a single node. Its strengths resemble Milvus’ strengths, with high performance on local tasks, good for prototyping, and it is open-source and widely adopted by the community. However, Milvus was chosen because of the potential continuation of this project, which ensures that the database does not need to be switched in the future, since Milvus can be deployed and scaled efficiently to meet growing demands. As mentioned above, a local instance of the Milvus database was sufficient for this project since it is only in an experimental phase and deployment of the RAG application is not planned as of now [56].

Furthermore, the same embedding model was used in all systems to ensure a fair evaluation across the different pipelines. The selection process involved reviewing the Hugging Face Massive Text Embedding Benchmark (MTEB) [20] leaderboard to identify the top-performing models in the Swedish language, while also providing a large context window and ensuring compatibility with the Milvus vector database. Additionally, the model’s size was considered, as we intended to run it locally on a desktop computer. Based on these criteria, BGE-M3 was selected as the embedding model for the three systems. With a context window of 8194 tokens and approximately 568 million parameters, it is sufficiently small to run locally [11]. The model ranked 13th on the MTEB leaderboard behind larger models with around 7 billion parameters, which were deemed too large. The only model within the same size range that ranked higher, *multilingual-e4-large-instruct*, was not considered due to its limitation of 512 tokens, which would truncate any chunks or queries exceeding this length [64].

Another shared feature was the prompt template for the models. The template shown in Figure 4 was used for the generation in all three pipelines. To get confident answers, we instructed the models to be experts using role-prompting, described in section 2.3.4. We chose to request concise answers to ensure that the output remained focused and relevant. If the model could not find the answer in the contexts, it should output “Jag har inte tillräcklig information för att besvara den frågan” to reduce the risk of hallucination.

```

1 [
2   {
3     "role": "system",
4     "content": "Du är en expert som svarar på frågor enbart
                  baserat på den givna kontexten med ett konsist svar. Om
                  informationen saknas i kontexten, svara 'Jag har inte
                  tillräcklig information för att besvara frågan'."
5   },
6   {
7     "role": "user",
8     "content": "Kontext:\n\n{context}\n\nFråga:\n\n{question}"
9   }
10 ]

```

Figure 4: Prompt-template used for generation in all three pipelines with the context and question added after retrieval.

4.2.2 Naive Pipeline

Apart from the shared features described above, the Naive RAG was a simple implementation of Retrieval-Augmented Generation. The preprocessed text was naively chunked by splitting the entire Markdown document, including tables, into chunks of 500 characters with a chunk overlap of 50 characters. The chunk size was arbitrarily set to 500 to create a baseline RAG pipeline for later evaluation. This also meant that the structure of the tables was not preserved since the chunking of the text could cut in the middle of a table. This approach relied on direct text retrieval from the vector database, where relevant context was retrieved based on the user query and later used to generate a response. The system did not include any complex pre- or post-processing of the query or the retrieved context, nor did it rely on additional components.

4.2.3 Semi-Structured Pipeline

The second constructed pipeline was built on the techniques of the naive pipeline, but involved more sophisticated chunking of the data and a document store to preserve the structure of the tables. The reason for having both a vector database and a document store was the need to handle different data types. The vector database enables efficient semantic search on textual data, but it can be difficult to query tables accurately using only text-based semantic search.

To create this document store, a new chunking method was implemented. This tech-

nique involved creating a novel approach to store the tables. To manage this, we extracted the JSON table objects from the Markdown file and stored each of the JSON objects as a separate file in local file storage, together with a unique ID. Given the experimental nature of the project, the need for a complex database setup was deemed unnecessary, and storing the tables in local files allowed for quicker implementation and easier iteration, while avoiding the overhead of managing a relational database. To enable similarity search for retrieving relevant tables, they were summarized into text using GPT-4o and stored in the vector database alongside regular text chunks. Each table summary was assigned the same unique ID as its corresponding table in the document store, so when the retriever selects a table summary, it is replaced with the actual table from the document store.

Additionally, the remaining text in the Markdown file was split at the largest headers, ensuring that each section of the PDF was treated individually and that no overlap between the sections occurred. The sections were then recursively chunked. As described in Section 2.3.1, recursive chunking is a technique that tries to split the data on given endpoints such as paragraphs, new lines, and punctuations. The chunk size was set to 1000 characters with an overlap of 100 characters, ensuring that the chunks were neither too small, like a single sentence, nor too large, such as a long paragraph. This balance allowed for more manageable data units while retaining context and coherence within each chunk.

4.2.4 Hybrid Pipeline

The third and most advanced pipeline we implemented was built on the semi-structured pipeline by leveraging the same chunking technique and document store, while introducing hybrid search in the retrieval step and re-ranking of the retrieved contexts. To enable hybrid search we leveraged the strength of the BGE-M3 embedding model to create both sparse and dense embeddings for all chunks and table summaries. The embeddings were stored in separate vector spaces in Milvus, each referencing the same chunk or table summary. This was because the sparse and dense embeddings had different dimensions and could not be stored in the same vector space. As outlined in Section 2.3.3, hybrid search combines both semantic and keyword-based retrieval by calculating the semantic similarity between the query and dense embedded chunks, as well as a relevance score based on keyword matches using sparse embeddings. This technique can capture both the contextual meaning and the exact terminology of the question.

After the hybrid retrieval, a ranking model was presented with the user question and the retrieved contexts, to sort and filter the contexts in order of relevance to the question. This step ensured that the most relevant contexts were sent to the generation step,

Table 1: Selected models and their parameter sizes.

Model	Parameter Size
GPT-SW3	6.7B
Llama 3.2	3B
DeepSeek-V3	617B

regardless of whether they had been retrieved through semantic similarity or keyword-matching. We chose to integrate the *bge-reranker-base* model in the hybrid pipeline to re-rank the retrieved contexts. It is a lightweight base model built with a cross-encoder architecture. With the retrieved context and the user question as input, it produces a score between 0 and 1 based on how similar they are and can re-rank the retrieved contexts based on that score. Since the model is a part of the BGE family, just as the embedding model, it enhanced the consistency between the retrieval and re-ranking steps. The model was also easily accessible through Hugging Face.

4.3 Selection of LLMs for GPT-SW3 Comparison

In alignment with Research Question 1 (Q1) in Section 1.2, two LLMs apart from GPT-SW3 were selected and evaluated within each of the pipelines described above. In selecting these models we considered multiple factors including parameter size, availability, and popularity. Today there is a vast amount of available generative models and we ended up choosing Llama 3.2 and DeepSeek-V3 to compare against GPT-SW3. These two models are different, as DeepSeek-V3 is a large and high-performing language model, while Llama 3.2 is a lighter model with fewer parameters than GPT-SW3. In Table 1 the chosen models and their number of parameters are presented.

While Llama 3.2 is a multilingual model, it does not officially support Swedish. This limitation made it interesting to compare it to GPT-SW3, which was developed for Nordic languages. Assessing the performance of a lightweight model like Llama 3.2, with only 3 billion parameters, on similar tasks provides valuable insight into how well GPT-SW3 performs. If a smaller model could achieve better results than GPT-SW3, there is no reason for users and companies to use the larger model with greater computational demands. On the other hand, we also wanted to benchmark GPT-SW3 against one large and high-performing LLM. DeepSeek-V3 stood out due to its high performance and the rapidly growing interest in the company since its release in late December 2024. If GPT-SW3 outperforms DeepSeek-V3, the argument for creating models for smaller languages would be stronger. According to AI Sweden, the developers of GPT-SW3, there is a growing interest and need for developing LLMs for smaller languages [19]. Therefore, a large model as DeepSeek-V3 is an interesting option for testing and com-

paring against GPT-SW3.

We accessed Llama 3.2 and DeepSeek-V3 through different APIs. Llama 3.2 can easily be called through the Hugging Face inference API. Hugging Face has collected many models in one API by hosting them at inference provider partners, so users can pay-as-you-go to Hugging Face and do API calls to models hosted by a third-party [31]. DeepSeek-V3, on the other hand, was called with DeepSeek’s API, as they host their models and offer an API with very competitive pricing compared to other companies with similar models. As GPT-SW3 was unavailable through any API, it was necessary to run the model locally or host it on a server. Since it demands a lot of GPU power, we ran it on Google Colab, a hosted Jupyter Notebook service that provides access to powerful GPUs [25]. With the paid “Pro” plan on Google Colab, we leveraged their most powerful GPU A100 to run GPT-SW3.

4.4 Evaluation

The evaluation of the systems was conducted using both RAGAs, an evaluation framework specifically for RAG systems, and human evaluation conducted by domain specialists. RAGAs provided us with quantitative insights on both the retriever and generator by automatically calculating the metrics described in the following sections using LLM-as-a-judge. Based on the results from RAGAs, the best pipeline was chosen as the base for human evaluation, where domain specialists ranked the answers of the three different LLMs. This qualitative assessment gave insight into the practical effectiveness of the system.

4.4.1 Test Dataset

To evaluate the performance of the RAG pipelines, a dataset containing questions and ground-truths was required. Since there existed no benchmark dataset in this specific domain, environmental certification in Swedish, a synthetic dataset was generated using an LLM. Although it would have been beneficial to have domain experts create the question-answer dataset, such resources were not available.

The synthetic dataset was generated with OpenAI’s GPT-4o model, a state-of-the-art language model showing impressive results in natural language understanding and text generation [5]. This model was chosen because of its performance and to avoid introducing bias in the evaluation by using one of the models that was being evaluated in this work.

Table 2: Breakdown of the final test dataset used in the RAGAs evaluation by context type and number of generated questions.

Context Type	Number of Generated Questions
1000-character snippet	97
Full-page document	18
Table-based content	27
Total	137

The first step of creating a synthetic dataset was to collect a diverse set of contexts to use as knowledge for question generation. These contexts consisted of different chunks from “Manual Miljöbyggnad Ombyggnad 4.0”, including 1000-character-long text chunks, full-page segments, and chunks that represent tables in the manual. The contexts were then embedded in tailored prompts, depending on the specific type of context, to guide GPT-4o in generating the questions. In the final step, the generated questions were prompted to the LLM, along with the related context as factual reference, to create ground-truth answers to the questions. The variation in size and content of the contexts, along with the use of different prompts, enabled the creation of a varied set of question-answer pairs, ranging from focused to general questions, as well as questions targeting the structural components of the manual.

Finally, a manual walkthrough of the question-answer pairs was done to ensure the quality of the dataset. Questions and answers that were ambiguous or inconsistent with their associated context were either revised or removed to ensure the creation of a high-quality benchmark dataset for evaluation. Table 2 shows the distribution of generated questions across different context types in the final test dataset.

4.4.2 Retrieval Metrics

For the evaluation of the retrieval step, context precision and recall were used. These metrics, referred to by different names in various RAG evaluation frameworks, provide valuable insights into how well contexts are retrieved and ranked [21] [53] [68].

Context Precision evaluates how well the relevant retrieved documents are ranked during the retrieval step. A search engine aims to present the most relevant match first, which is what context precision assesses. It is defined as

$$\text{Context Precision} = \frac{\sum_{k=1}^K p(k) \cdot \mathbb{1}_v(k)}{\sum_{k=1}^K \mathbb{1}_v(k)},$$

where $p(k)$ is the precision at rank k , and $\mathbb{1}_v(k)$ is an indicator function:

$$\mathbb{1}_v(k) = \begin{cases} 1 & \text{if the context at rank } k \text{ is relevant,} \\ 0 & \text{otherwise.} \end{cases}$$

The denominator represents the total number of relevant contexts in the top K results. Precision at rank k is the cumulative sum of relevant documents at rank k , defined as

$$p(k) = \frac{\sum_{n=1}^k \mathbb{1}_v(n)}{k}.$$

Context Recall measures how accurately the retrieved contexts capture the information in the ground truth. The ground truth consists of claims, and context recall evaluates the proportion of these claims present in the retrieved contexts. Context recall is expressed as

$$\text{Context Recall} = \frac{\sum_{c \in C_{\text{GT}}} \mathbb{1}_{RC}(c)}{|C_{\text{GT}}|}$$

where $\mathbb{1}_{RC}(c)$ is an indicator function:

$$\mathbb{1}_{RC}(c) = \begin{cases} 1 & \text{if claim } c \in RC, \\ 0 & \text{otherwise.} \end{cases} \quad (1)$$

and RC is the context(s) that the system retrieved.

Here, C_{GT} represents the set of all claims in the ground truth, and $|C_{\text{GT}}|$ denotes its cardinality.

4.4.3 Generation Metrics

The evaluation of the generation stage of the pipeline primarily assesses the ability of a LLM to generate accurate responses based on the context provided in the prompt. This evaluation was also done using metrics from the RAGAs library, namely faithfulness and answer relevance.

Faithfulness evaluates how factually consistent the response is with the retrieved context. It measures the ratio of claims in the response that is supported by the retrieved

context. The score is between 0 and 1, where a higher score indicates a more faithful answer. It is calculated by

$$\text{Faithfulness} = \frac{\sum_{c \in C_r} \mathbf{1}(c)}{|C_r|},$$

where C_r is the set of all claims in the response and $s(c)$ is the same indicator function used for context recall illustrated in Equation (1).

The **Answer Relevance** metric examines the relevancy of the response based on the user query. This metric can be seen as an end-to-end evaluation of the system since it first relies on the retriever to collect the correct context and then the generating LLM to understand the question and use the context to answer the question factually. To calculate the answer relevance, N artificial questions are generated based on the response, and cosine similarity is calculated between the embedded user query (E_u) and the embedded artificial questions (E_{a_i}). The average of these scores is the answer relevance. It is expressed as

$$\text{Answer Relevance} = \frac{1}{N} \sum_{i=1}^N \text{CosineSimilarity}(E_u, E_{a_i}),$$

where $\text{CosineSimilarity}(E_u, E_{a_i})$ is a function that calculates the cosine similarity between the embedded user query E_u and the i -th artificial question E_{a_i} , and N is the total number of artificial questions generated. Cosine similarity is a similarity measure of two non-zero vectors, A and B, and is calculated as

$$\text{CosineSimilarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

4.4.4 RAGAs Experiments

The evaluation of the implemented pipelines in combination with the selected LLMs was conducted by leveraging the RAGAs framework, explained in Section 2.4. The four metrics presented in the previous sections were calculated using an LLM-as-a-judge approach, where we leverage the strength of an LLM to perform nuanced evaluation, which typically requires human judgment.

When selecting an LLM to serve as the judge, we deliberately chose a model different from those evaluated in this paper. This was deemed necessary to minimize bias and

to achieve an objective assessment. Furthermore, the model selection was based on three additional criteria: how it is accessed, its performance, and the price of running it. The first criterion was important since the evaluation required substantial computation. With each metric requiring multiple calls to the LLM, a fast model would be necessary to complete the evaluation. The second factor, performance, refers to the models' ability to interpret the questions, answers, and contexts – specifically in Swedish. Since a weak model would not produce reliable results, we needed a capable judge to compute each metric. The last criterion was also crucial because of the many requests the model would handle when running the evaluation. More capable models are often more expensive, and a balance between performance and cost was needed. This evaluation led to the selection of OpenAI's model GPT-4o-mini, as it meets all three criteria. It is easily accessed through OpenAI's API, offers strong performance [44], and is a good tradeoff between performance and cost, being almost 17 times cheaper than its more powerful parent model GPT-4o [45].

The test dataset was used to evaluate the performance of the systems and the LLMs. The process involved calculating the metrics for each of the 137 question-answer pairs in the dataset and then averaging over all scores. To better understand the LLM's abilities to extract useful information from varying numbers of contexts, we evaluate each pipeline and accompanying LLM with different K -values. This K -value is the number of context chunks retrieved and sent to the LLM for generation. We chose to evaluate on $K = \{1, 3, 5, 7, 9\}$ to get an understanding of how sensitive each pipeline is to the number of contexts retrieved, and whether the performance increases, plateaus, or degrades as more information is presented. This resulted in $5 * 3 * 3 * 4 = 180$ different data points being calculated, corresponding to 5 K -values, 3 pipelines, 3 LLMs, and 4 metrics evaluated. With each data point being an average of 137 question-answer pairs, this shows the scale and depth of the evaluation, providing meaningful insight into the performance across the entire dataset. Figure 5 depicts an example object used for evaluation, formatted according to the structure required by the RAGAs framework. Each object contains the ```user_input``` and ```ground_truth``` from the test dataset, as well as the ```retrieved_contexts``` and ```response``` generated in the pipeline.

```

1 {
2   "user_input": "Vilka akustiska parametrar bedöms i
3     Miljöbyggnad 4.0 för att säkerställa en god ljudmiljö?",
4   "retrieved_contexts": [
5     "INDIKATOR 6 LJUD: # INDIKATOR 6 LJUD\n## Syfte\nSyftet är
6       att premiera byggnader som projekteras, byggs och
7       förvaltas för god ljudmiljö.\n## Vad bedöms\nLjudmiljön
8       enligt BBR och de svenska ljudstandarderna samt
9       förvaltningsrutiner.\n## Globala mål och Sveriges
10      miljömål\n## Instruktion\nI Miljöbyggnad bedöms
11      ljudmiljön med följande akustiska parametrar:\n-
12      ljudtrycksnivå inomhus från installationer\n-
13      luftljudsisolering / ljudnivåskillnad\n-
14      stegljudsnivå\n- ljudtrycksnivå inomhus från yttre
15      ljudkällor\n- rumsakustik (enbart SS 25268:2022).\nVal
16      av parameter görs utifrån vilka som bedöms få störst
17      påverkan från brukare på betygsnivå Silver."
18   ],
19   "response": "I Miljöbyggnad 4.0 bedöms följande akustiska
20     parametrar: ljudtrycksnivå inomhus från installationer,
21     luftljudsisolering/ljudnivåskillnad, stegljudsnivå,
22     ljudtrycksnivå inomhus från yttre ljudkällor och
23     rumsakustik (enbart SS 25268:2022).",
24   "ground_truth": "I Miljöbyggnad 4.0 bedöms följande
25     akustiska parametrar för att säkerställa en god
26     ljudmiljö: ljudtrycksnivå inomhus från installationer,
27     luftljudsisolering, stegljudsnivå, ljudtrycksnivå
28     inomhus från yttre ljudkällor och rumsakustik."
29 }

```

Figure 5: An example object with $K = 1$ used for evaluation with the RAGAs framework. It contains the user input question, the retrieved context from the knowledge base, the generated response from the system, and the ground truth answer used for scoring.

4.4.5 Human Evaluation

In addition to the automated evaluation with a synthetic dataset, a human evaluation was conducted by domain experts. Three *Certification Specialists* from SGBC were invited to test the system with tailored questions, grading the response from the best performing RAG pipeline based on the RAGAs evaluation. The human evaluation aims to complement the automated evaluation by providing insights in which of the LLMs are most preferred by human users, as well as to assess the practical applicability of the

complete system. The evaluation process was designed to simulate the real use of the system and consisted of three different phases:

First, the three specialists were asked to create ten questions each that they would expect the system to answer effectively. The specialists did not receive any specific guidance on how to write the questions in order to avoid introducing bias. Thus, the result was a mix of questions from their daily work, general questions about “Miljöbyggnad”, and specific questions on “Manual Miljöbyggnad Ombyggnad 4.0”. However, the answers to some of the collected questions could not be found in “Manual Miljöbyggnad Ombyggnad 4.0”. Therefore, those questions were removed to test the systems only on questions related to the data in the manual. This collection process resulted in 23 diverse questions from separate sources that lay the foundation for the human evaluation.

Second, the collected questions were processed using the best-performing system, as determined by the RAGAs evaluation described in the previous section. Each question was processed through the pipeline three times, switching the generating LLM each run. This resulted in three distinct answers per question, one from each of the LLMs being evaluated.

Lastly, an online form with the questions and the corresponding answers was created to collect feedback from the specialists on how they perceived the quality and usefulness of each response. Feedback was collected by presenting each question to the evaluators and asking them to rank the three corresponding answers in order of perceived correctness and fluency. Using this method, we obtained a rank for each LLM for every question, ensuring a transparent and effective comparison of their performance from a human perspective.

5 Results

This section presents results from the evaluation of our Retrieval-Augmented Generation systems. First, we analyze the retrieval performance of the three pipelines using precision and recall. Next, we assess the output generation of LLMs in terms of faithfulness and relevance. We then compare the performance of the LLMs across the pipelines to answer the research questions. Finally, the results of a human evaluation performed by experts from the Sweden Green Building Council are presented.

5.1 Retrieval Results

The retrieval performance of the three pipelines described in Section 4.2 was evaluated using context precision and recall. These metrics assess how effectively each pipeline retrieves relevant information from the vector database in response to the user query. The retrieval results aim to compare the three pipelines to determine which can retrieve the best contexts.

Figure 6 presents the context precision results for the three pipelines across varying values of K . It can be noted that the semi-structured and hybrid pipelines perform similarly in presenting the most relevant contexts first, while the naive pipeline struggles when retrieving fewer contexts. The decrease in precision when increasing the value of K is explained by the fact that retrieving more contexts increases the likelihood of introducing irrelevant information. However, a notable point in this figure is that the hybrid pipeline decreases at the same rate as the semi-structured pipeline, despite leveraging a re-ranking model to mitigate this decrease.

Figure 7 presents the corresponding context recall results, providing insight into each pipeline’s ability to retrieve contexts supporting the ground-truth claims. These results indicate that the semi-structured and hybrid pipelines, which both leverage the same indexing technique, perform similarly, while the naive pipeline performs worse. An increase in performance with growing K is expected since the likelihood of retrieving all the claims in the ground truth rises with the number of contexts retrieved.

The retrieval results in Figures 6 and 7 show that the semi-structured and hybrid pipeline outperforms the naive pipeline, for almost all K , in retrieving relevant information to a query when the value of K is less than 10. This finding indicates that a more structured indexing of the data, as done in the semi-structured and hybrid pipelines, correlates with retrieving and ranking relevant contexts from the database.

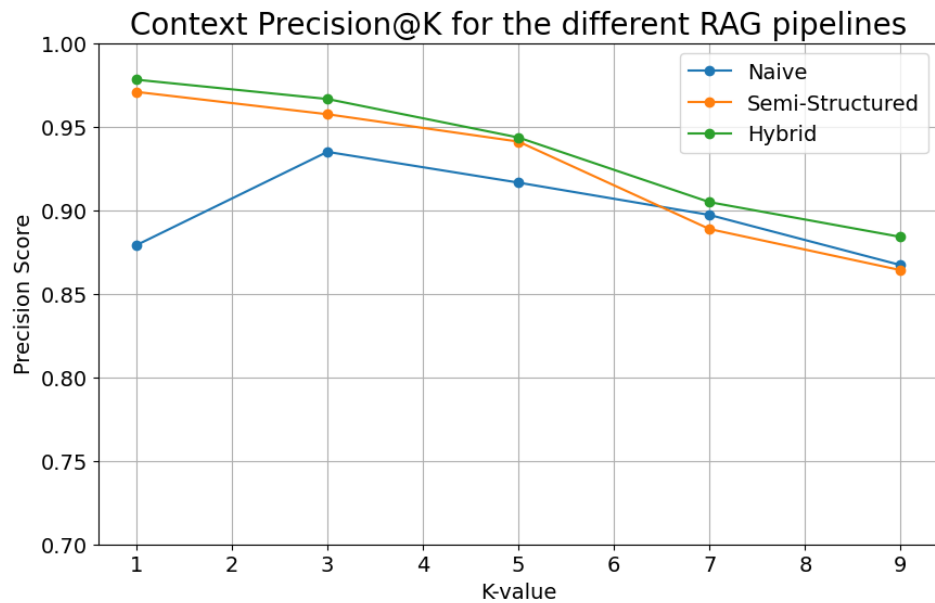


Figure 6: Context precision of the three pipelines across varying values of K . Higher precision indicates a greater proportion of relevant contexts among the top K retrieved results.

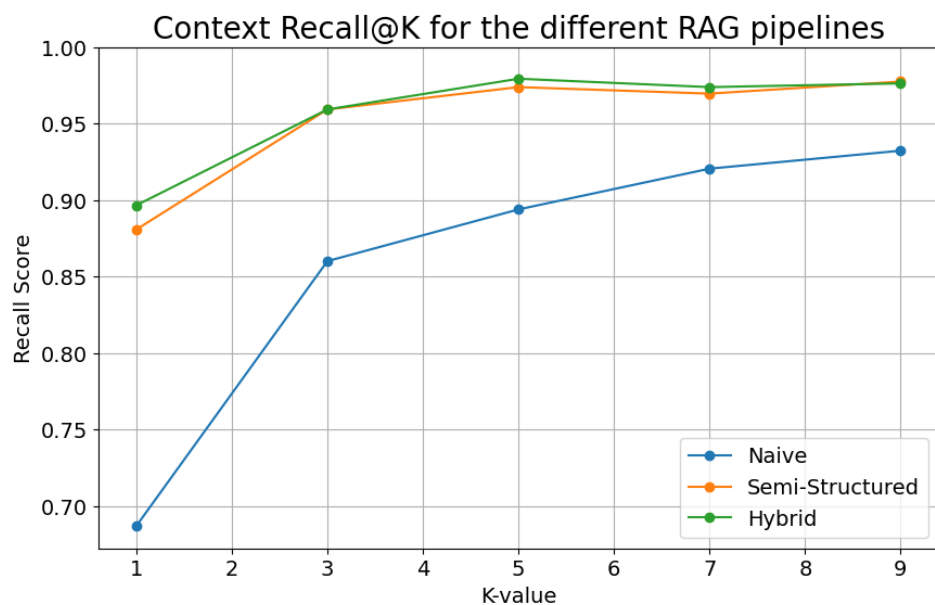


Figure 7: Context recall of the three pipelines across varying values of K . Higher recall indicates that a greater proportion of the retrieved contexts can be used to answer the query.

5.2 Generation Performance Across Pipelines

To evaluate the end-to-end performance of the RAG systems, we assess how each of the three LLMs perform when paired with the different pipelines. The results presented in this section connect to Research Question 2 (Q2) about how the performance changes when the RAG system is made more advanced. To evaluate this, we plot how each LLM performs in each of the three pipelines for different values of K . The results are computed using RAGAs and our generated test dataset. In Figure 8 and 9, faithfulness and answer relevance for GPT-SW3 in the naive, semi-structured, and hybrid systems are visualized for increasing K from 1 to 9. We notice that the semi-structured and hybrid systems perform similarly, with scores that increase together with K , while the naive system performs worse. The naive system does not achieve better performance with more contexts, instead decreasing the faithfulness score with a larger K .

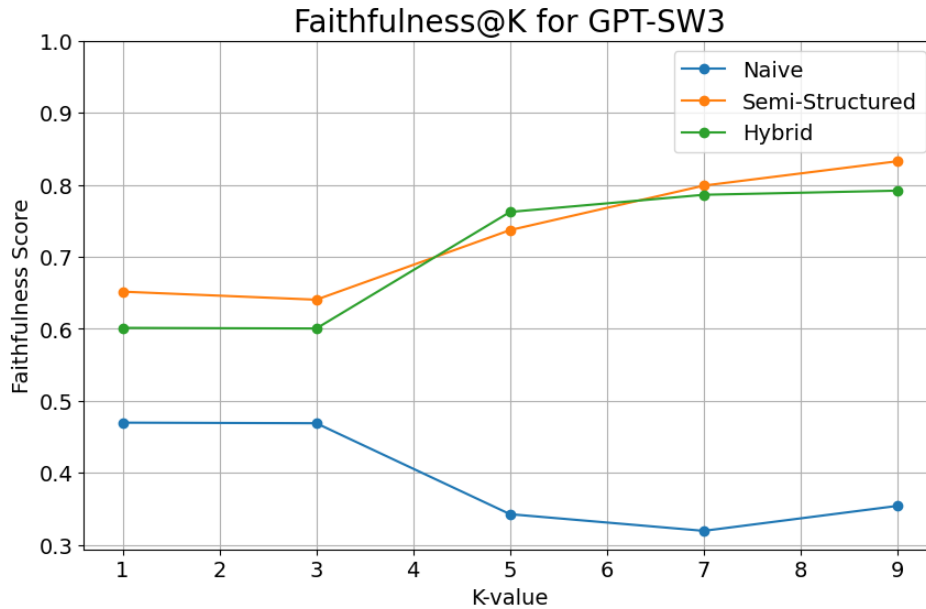


Figure 8: Comparison of faithfulness scores across different RAG pipeline configurations (naive, semi, and hybrid) for GPT-SW3 at various values of K . The plot illustrates how the faithfulness scores change with different RAG approaches.

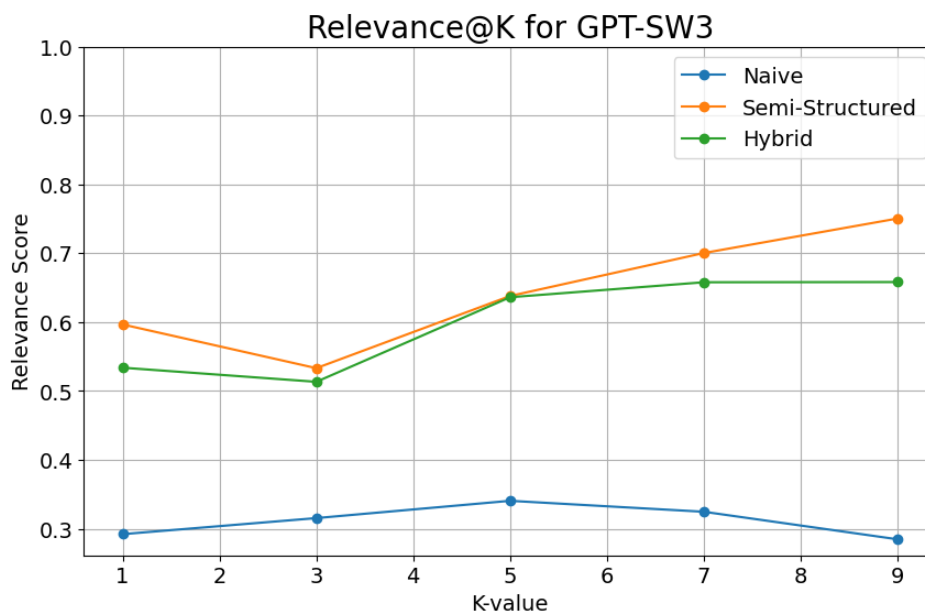


Figure 9: Comparison of answer relevance scores across different RAG pipeline configurations (naive, semi, and hybrid) for GPT-SW3 at various values of K . The plot illustrates how the answer relevance scores change with different RAG approaches.

In Figure 10 and 11, the faithfulness and relevance results for Llama 3.2 are visualized. The difference between the three pipelines is much smaller than for GPT-SW3, although we notice that the naive pipeline yields low scores for $K = 1$ for both metrics. Overall, Llama 3.2 achieves significantly better results than GPT-SW3 with the best scores for both relevance and faithfulness at $K = 7$ in the hybrid pipeline.

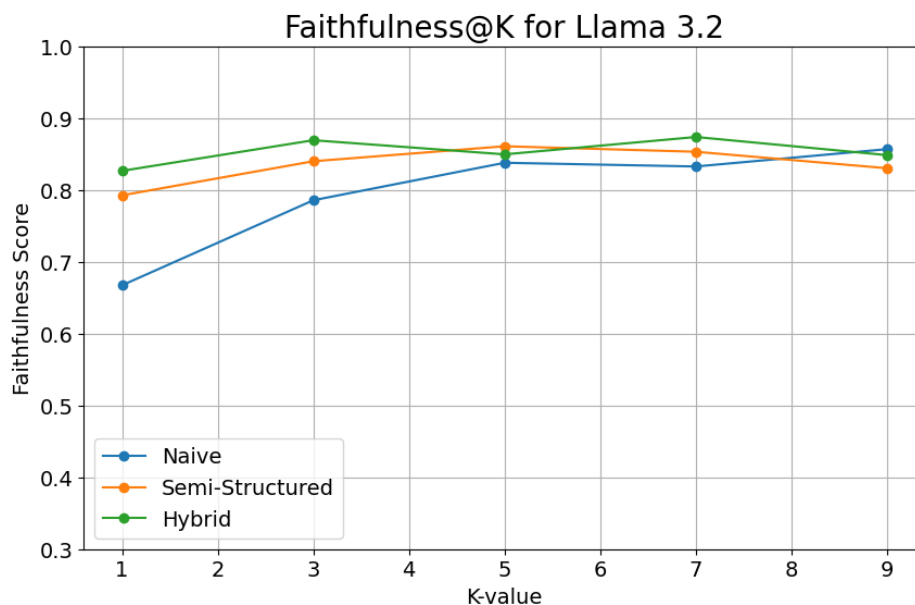


Figure 10: Comparison of faithfulness scores across different RAG pipeline configurations (naive, semi, and hybrid) for Llama 3.2 at various values of K. The plot illustrates how the faithfulness scores change with different RAG approaches.

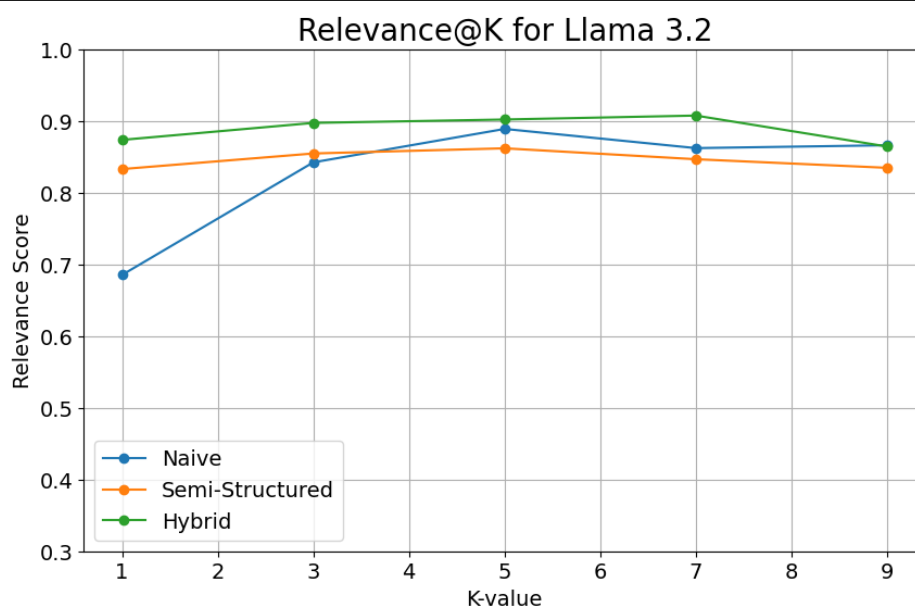


Figure 11: Comparison of answer relevance scores across different RAG pipeline configurations (naive, semi, and hybrid) for Llama 3.2 at various values of K. The plot illustrates how the answer relevance scores change with different RAG approaches.

The results for the last LLM, DeepSeek-V3, are shown in Figure 12 and 13. Similarly to Llama 3.2, the scores for the naive pipeline with $K = 1$ are low, but then the faithfulness and relevance scores of the naive pipeline increase significantly and reach the same levels as the semi-structured pipeline and even better than the hybrid pipeline for larger K values. We achieve the best overall results with the semi-structured pipeline for faithfulness and relevance. However, one notable result is that the naive pipeline receives a higher faithfulness score than the other two pipelines for $K = 7$ and $K = 9$.

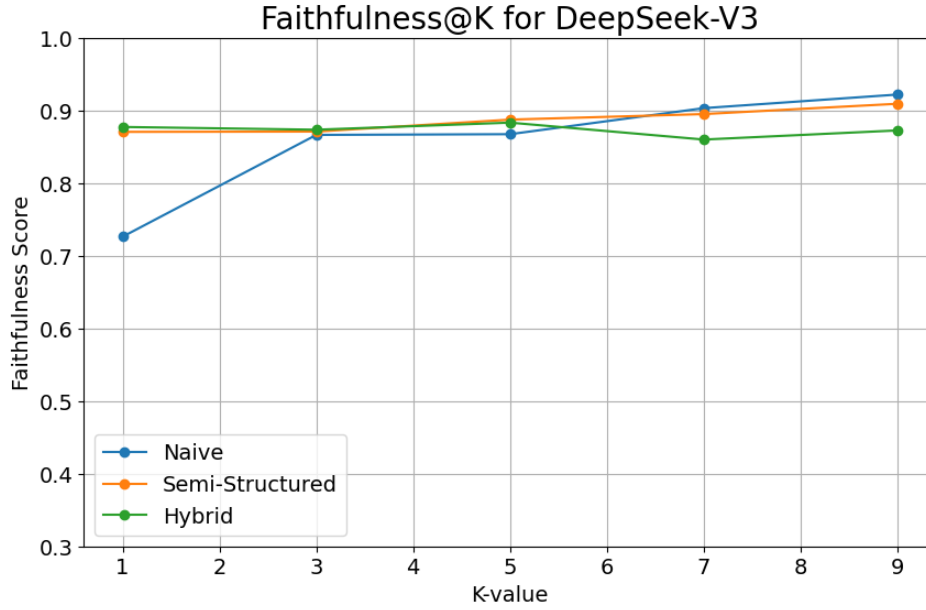


Figure 12: Comparison of faithfulness scores across different RAG pipeline configurations (naive, semi, and hybrid) for DeepSeek-V3 at various values of K . The plot illustrates how the faithfulness scores change with different RAG approaches.

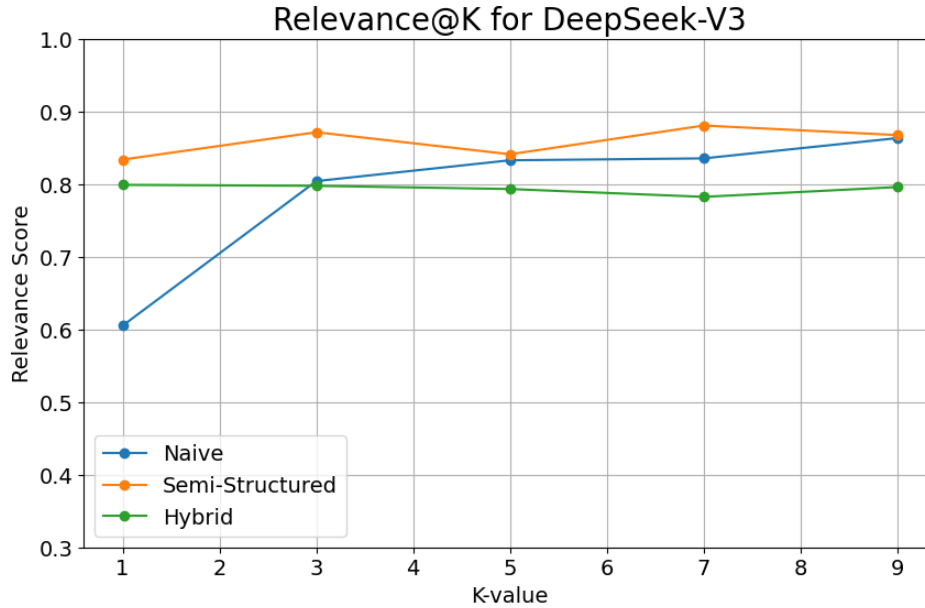


Figure 13: Comparison of answer relevance scores across different RAG pipeline configurations (naive, semi, and hybrid) for DeepSeek-V3 at various values of K . The plot illustrates how the answer relevance scores change with different RAG approaches.

5.3 Performance of Large Language Models

The performance of the LLMs used as generators differs between the three pipelines. As the pipelines retrieve different contexts, the prompts to the LLM vary, and that affects the performance. By observing the relevance and faithfulness metrics, we can see how the LLM generates an answer based on the given context. In Figure 14, the faithfulness for the three LLMs in our semi-structured system is shown. GPT-SW3 performs the worst but increases the most with a growing K value. DeepSeek-V3 achieving the highest scores overall K is reasonable, as the LLM is much larger than the other two. However, Llama 3.2 is not far from DeepSeek-V3, and Figure 15 shows that it has better relevance than DeepSeek-V3 for $K = 5$. In addition to that, DeepSeek-V3 outperforms both Llama 3.2 and GPT-SW3 in terms of relevance and faithfulness in the semi-structured pipeline. We chose to present the comparison of LLMs in the semi-structured pipeline, as the results from the other pipelines were largely consistent and showed similar trends. These graphs can be seen in Figures A.1 - A.4 in Appendix A.

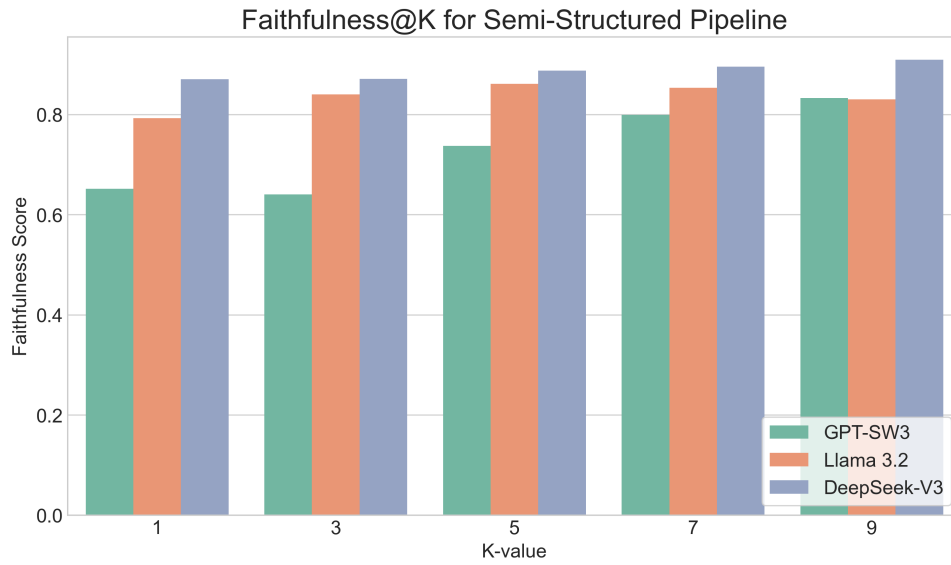


Figure 14: Comparison of faithfulness scores across three LLMs (GPT-SW3, Llama 3.2, and DeepSeek-V3) in the semi-structured RAG pipeline, evaluated at various K-values (1, 3, 5, 7, and 9). The plot illustrates how the faithfulness scores of each LLM vary with changes in K-value within the semi-structured configuration.

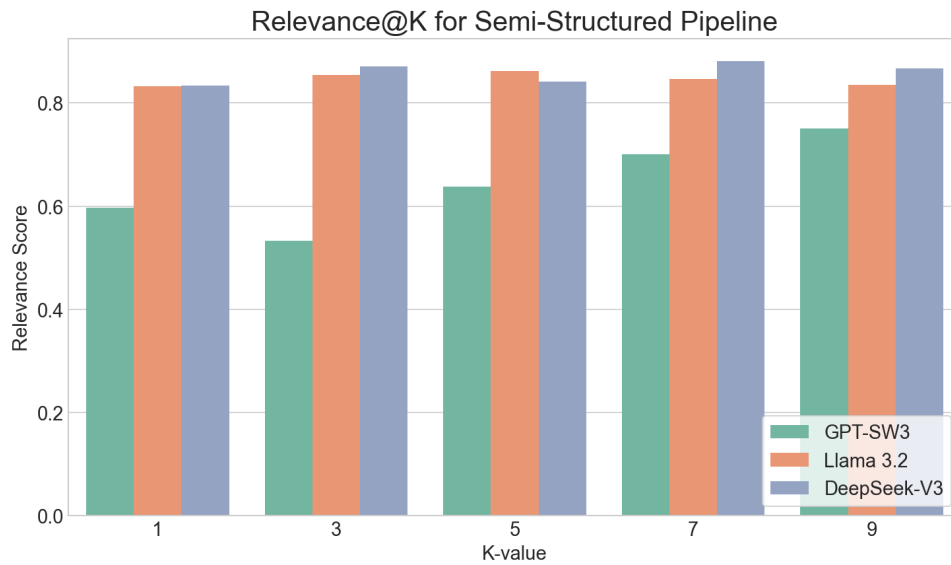


Figure 15: Comparison of answer relevance scores across three LLMs (GPT-SW3, Llama 3.2, and DeepSeek-V3) in the semi-structured RAG pipeline, evaluated at various K-values (1, 3, 5, 7, and 9). The plot illustrates how the answer relevance scores of each LLM vary with changes in K-value within the semi-structured configuration.

In Table 3, a question and ground-truth answer from our test dataset, along with responses from the three LLMs are presented. This displays the differences and similarities in responses between the LLMs. In this example, Llama 3.2 and DeepSeek-V3 answer the question correctly and are very similar to the ground truth. GPT-SW3, on the other hand, answers “Jag har inte tillräcklig information för att besvara frågan”, which is the answer it is prompted to generate if it cannot answer the given question with the retrieved context. Even if the LLM has a lot of information in the context, it may not be able to understand it and therefore generate incorrect answers. In this case, as both Llama 3.2 and DeepSeek-V3 managed to answer almost identically, good context was probably retrieved. One possible explanation for why GPT-SW3 failed is that the answer in the context might have been in a table, as the grades “GULD-betyg” are presented within tables in the manual. The table structure can be difficult to comprehend for LLMs, and they may misinterpret or neglect the information.

Table 3: Example question with ground-truth and the answers from the three LLMs that are being evaluated.

Question	Vilka krav måste uppfyllas för att en bostad ska kunna erhålla GULD-betyg enligt indikator 3 för energianvändning?
Ground Truth	För att en bostad ska kunna erhålla GULD-betyg enligt indikator 3 för energianvändning måste Betyg Silver uppfyllas, 70 procent av BBR:s energikrav uppfyllas, energideklaration med radonmätning som understiger 100 Bq/m ³ finnas, samt att mer än 5 procent av fastighetsenergin är lokalt genererad, förnybar och flödande.
DeepSeek-V3	För att en bostad ska erhålla GULD-betyg enligt indikator 3 för energianvändning måste betyg Silver uppfyllas, energikravet ska vara 70 procent av BBR:s krav, energideklarationen måste visa radonmätning under 100 Bq/m ³ , och minst 5 procent av fastighetsenergin ska vara lokalt genererad, förnybar och flödande.
Llama 3.2	För att en bostad ska kunna erhålla GULD-betyg enligt indikator 3 för energianvändning måste följande krav uppfyllas: • Betyg Silver uppfylls. • 70 procent av BBR:s energikrav uppfylls. • Energideklaration med radonmätning som understiger 100 Bq/m³. • > 5 procent av fastighetsenergin är lokalt genererad, förnybar och flödande.
GPT-SW3	Jag har inte tillräcklig information för att besvara frågan.

5.4 Results From Human Evaluation

Based on the RAGAs evaluation discussed in the previous sections, the semi-structured pipeline, with five retrieved contexts, was selected for the human evaluation. The expert-generated questions were answered using the semi-structured RAG system, in combination with each LLM, and the results of this evaluation are summarized in Table 4. For each question, the experts ranked the three answers, one from each LLM, without knowing which LLM generated it. DeepSeek-V3 is the most preferred generator, with almost 70% being the best alternative. Llama 3.2 and GPT-SW3 had almost the same number of top choices, but Llama 3.2 had more second places, while GPT-SW3 dominated the last choice. Notable is that DeepSeek-V3 is much better than Llama 3.2 according to the human evaluation, which differs from the RAGAs evaluation seen in the previous section, where Llama 3.2 and DeepSeek-V3 have very similar scores.

Table 4: Human evaluation results comparing GPT-SW3, Llama 3.2, and DeepSeek-V3 based on preference rankings.

Model	Top Choice (%)	Second Choice (%)	Last Choice (%)
GPT-SW3	14.5%	20.3%	65.2%
Llama 3.2	15.9%	52.2%	31.9%
DeepSeek-V3	69.6%	27.5%	2.9%

6 Discussion

In this section, we reflect on the results presented in Section 5, relating them to our research questions. We also consider the broader impacts of our work, the strengths and limitations of our chosen methods, and discuss potential improvements.

6.1 Discussion of Results

Based on the results in the previous section, we can draw several conclusions related to our research questions presented in Section 1.2.

6.1.1 Research Question 2

To investigate whether the performance of the question-answering systems changes with increasing complexity in the pipelines (Research Question 2), we developed and evaluated three different RAG systems with varying structures. The context precision and recall results from the three pipelines, as shown in Figures 6 and 7, are particularly relevant to this question. Aside from the fact that the systems differ in complexity, the graphs reveal differences in the retrieval performance between the pipelines, confirming their suitability for addressing Research Question 2.

The precision and recall graphs show that the naive pipeline achieves a lower score than the other pipelines. A plausible explanation for this is the different indexing techniques the pipelines leverage. The naive pipeline chunking uses a chunk size of 500 characters and does not take different sections or tables in the data into account. This can result in chunks that cut in the middle of sentences and split tables in half. The more advanced pipelines, semi-structured and hybrid, leverage recursive chunking with a 1000-character chunk size, described in Section 2.3.1, as well as extracting the tables in the manual as separate chunks, to retain the semantic structure of the chunks. Another interesting finding in the retrieval results is that the semi-structured and hybrid pipelines achieve very similar scores. A possible reason for this result is that the hybrid pipeline can retrieve overlapping chunks when using both dense and sparse retrieval. Since both retrieval methods operate on the same set of chunks, they may retrieve the same context if it scores highly on both semantic similarity (dense) and keyword-based matching (sparse). This leads to a duplication of contexts, and no new information is retrieved by leveraging sparse embeddings. The semi-structured pipeline thus achieves a similar context recall score by only using dense embeddings and leveraging similarity search. This explanation is backed up by the context recall results, which indicate that the semi-

structured and hybrid pipelines fetch almost all the relevant context for large values of K , possibly the same set of contexts. The hybrid pipeline earns a slightly better context precision score plausibly by leveraging a re-ranker model that can change the order the contexts are presented, thus obtaining a better precision. Moreover, the semi-structured and hybrid pipelines achieve relatively high scores for $K = 1$, indicating that even if only a singular context is retrieved, it matches the user query well. This finding supports the claim that the implemented chunking technique for these pipelines works well.

To gain a more comprehensive understanding of how pipeline complexity impacts overall question-answer performance, we extend our analysis to include the generation outputs from each pipeline. By examining the faithfulness and answer relevance results of the three LLMs across the different pipelines, as presented in Figures 8–13, we can further investigate Research Question 2. A general trend observed across these figures is that the naive pipeline consistently yields the lowest performance across all three models for $K = 1$. Again, this is plausibly due to the chunk size of the naive pipeline, where the smaller chunk size limits the amount of text in a chunk, and when $K = 1$, it cannot retrieve enough relevant information to answer the user’s question correctly. In contrast, the semi-structured and hybrid pipelines demonstrate significantly higher performance for smaller K , and increase marginally across most K values for all models. Interestingly, there is no clear winner between the semi-structured and hybrid pipelines, as both achieve top scores depending on the model and evaluation metric. When examining the individual models, we observe that all three pipelines perform worse when paired with GPT-SW3. Notably, the combination of GPT-SW3 and the naive pipeline struggles badly with faithfulness and relevance scores consistently below 0.5 for all K -values. A plausible explanation for this is that GPT-SW3 is bad at filtering out information in an unstructured context. In contrast, when the naive pipeline is paired with Llama 3.2 and DeepSeek-V3, it struggles with smaller K -values but improves significantly to match the performance of both the semi-structured and hybrid pipelines for large K -values. This increase indicates that Llama 3.2 and DeepSeek-V3 are more effective at finding the relevant information in large contexts, even if it is unstructured.

In summary, the findings related to Research Question 2 indicate that increasing the complexity of the RAG pipeline can lead to improved overall performance of the question-answering system. The most notable improvement was observed when using GPT-SW3, where the naive pipeline performed significantly worse than the semi-structured and hybrid alternatives. However, when pairing the pipelines with Llama 3.2 or DeepSeek-V3, the performance gap across the pipelines was smaller, particularly for higher K -values, suggesting that the choice of generation model plays a critical role in determining the effectiveness of a RAG system, regardless of pipeline complexity.

6.1.2 Research Question 1

In order to answer Research Question 1, we leverage the results in Section 5.3, which show a comparison between the models in the semi-structured pipeline. As shown in Figures 14 and 15, DeepSeek-V3 consistently outperforms the other models across nearly all values of K , with Llama 3.2 as a close second and GPT-SW3 performing the worst. These findings are particularly interesting given the significant differences in model size: Llama 3.2 is the smallest with 3 billion parameters, GPT-SW3 has 6.7 billion, and DeepSeek-V3 is considerably larger with 617 billion parameters. However, when examining the training data of the models, as described in Section 2.1, a correlation with the observed performance emerges. GPT-SW3 was trained on 320 billion tokens, predominantly in Swedish, whereas Llama 3.2 and DeepSeek-V3 were trained on substantially larger datasets containing 9 trillion and 14.8 trillion tokens, respectively. This suggests that the model size alone does not account for performance differences and that training data plays a role in the performance of the models, regardless of which languages the models are trained on. Interestingly, GPT-SW3 performs relatively well on the evaluated task when considering the ratio between the amount of training data and its resulting performance compared to the other models.

To further analyze Research Question 1, we examine the example model output presented in Table 3. These responses clearly illustrate why GPT-SW3 receives a lower score than Llama 3.2 and DeepSeek-V3 in the RAGAs evaluation. As outlined in Section 5.3, both Llama 3.2 and DeepSeek-V3 produce accurate and complete answers that align with the ground truth. In contrast, GPT-SW3 fails to generate a response, indicating insufficient information in the context. The two correct answers show that contexts containing the correct information have been retrieved, and GPT-SW3 simply fails to leverage the context to answer the question. This example shows the superiority of Llama 3.2 and DeepSeek-V3 over GPT-SW3 in their ability to effectively find information in large contexts and use it for generation. In contrast, GPT-SW3 falls short on this task, highlighting its limited ability to interpret and utilize information in a RAG system.

For the human evaluation, we decided to use the semi-structured pipeline as it got promising results in the RAGAs evaluation. As the hybrid and semi-structured pipelines produced similar results, we would have wanted to evaluate both systems with experts, but as we had limited resources, we chose to do a more extensive human evaluation on one of the pipelines and decided on the semi-structured arbitrarily. We decided to use five retrieved contexts as it is a good trade-off between context precision and recall, as shown in Section 5.1. The evaluation provides insights into how the model outputs are perceived by human evaluators, thereby contributing to an understanding of the model’s practical usefulness in the system. The first impression of Table 4 is that it correlates

with the semi-structured results from RAGAs presented in Section 5.3 as DeepSeek-V3 is best, Llama 3.2 second, and GPT-SW3 worst. But as we only rank the models from best to worst, we might miss information. Seen in Figure 14 and 15, Llama 3.2 and DeepSeek-V3 produce almost similar scores, but in the human evaluation, DeepSeek-V3 is higher rated. A plausible explanation for this is that Llama 3.2 and DeepSeek-V3 give answers that contain the correct information, but DeepSeek-V3 formulates its responses more naturally and appealingly to human evaluators. An alternative evaluation method could be to rate each answer on a scale instead of against each other. This could give deeper insights into how each model performs according to human experts. We chose not to do that type of ranking because it demands more of the experts, as it might be difficult to know what a perfect answer is. It is more intuitive to compare answers against each other and determine a ranking from most to least preferable.

In summary, combining the results from RAGAs, the example model output, and the human evaluation, a clear conclusion emerges regarding Research Question 1: GPT-SW3 can not compete with other state-of-the-art LLMs within the evaluated RAG systems. This result yields an important question regarding the value of developing domestic language models like GPT-SW3 trained on predominantly Swedish text. While GPT-SW3 demonstrates the potential of a Swedish-trained language model, its inability to match the performance of more broadly trained models, regardless of their size, suggests that it is currently not competitive from a commercial perspective. Improving performance would likely require access to significantly more training data, which is a challenge for smaller languages like Swedish. However, the development of domestic language models still holds strategic and research value. It contributes to a deeper understanding of language models, promotes linguistic and cultural inclusion, and fosters national competence in AI development. Additionally, it ensures that smaller languages, like Swedish, are not excluded from future advancements in Natural Language Processing.

6.2 Discussion of Methods

In this section, we review parts of our method, including the ingestion pipeline and the evaluation process. We discuss the trade-offs involved in our implementation, limitations, and possible improvements.

As described in Section 4.1, our PDF ingestion is based on screenshots and image prompting to GPT-4o and demands manual correction. This solution did yield the best results for our data, as it required the least manual correction compared with the other alternatives. The RAG pipelines in this study are based on the “Manual Miljöbyggnad Ombyggnad 4.0”. However, the system could be further developed by utilizing additional manuals or documents, which would broaden the range of questions the system

could handle. A problem with our implementation is that the ingestion of new data is not that effective due to the processing of the PDF as images. If the manuals had a more suitable table structure, uploading the data into our vector database would be much more effective and require less or no manual correction. If the tables had more basic structures and no icons that could mislead parsing models, it would probably be better to use LlamaParse or Unstructured, as those frameworks were faster and easier to use than the solution we ended up with. Along with adding new data, a possible extension of the system would be to add metadata in the indexing step. This could add more information to each context, as the metadata can explain which manual and page the context belongs to. As we only used one manual, the implementation of metadata did not seem vital. But a system based on multiple manuals would need the manual version metadata to know from which manual it should retrieve information. Additionally, our ingestion implementation made it difficult to save metadata such as page number, which led us to decide not to implement it in this system. A system based on multiple manuals would also put higher demands on the user's question, as the users would have to specify which manual the question is about.

In Section 2.3.4 we outline techniques for improving the RAG. However, query optimization was not used in any of the three pipelines. This technique was disregarded due to the nature of our test dataset and the questions we received from the domain experts. Since all these questions were well formulated and correct, we did not find it necessary to implement a step that would compress or rewrite the questions. Considering this, it would likely be a relevant technique if the system were to be put in production. This could improve the system by rephrasing or merging sparse user questions into more informative and complete ones, making it easier for the model to retrieve relevant information.

Another important part of our method and research is the RAGAs evaluation. In Section 4.4.4, we motivate the choice of GPT-4o-mini as our LLM-as-a-judge. However, regardless of which model is used, there might be potential problems with LLM-as-a-judge. Since RAGAs metrics rely on different inputs, for example, the retrieved context, the output, or both, the evaluation depends on the model accurately interpreting all of that information. In practice, we cannot be sure that the LLM understands everything in the context, especially when it is long, noisy, or unstructured. This can lead to unreliable scores that do not reflect the true quality of the response. Ideally, the LLM used as a judge would process and evaluate the context and answer as accurately as a human expert. Although human evaluation would be preferable to assess the quality of output, it is not feasible on our scale, with more than 24,000 evaluations in total. Additionally, the RAGAs evaluation is based on the generated test dataset. As described in Section 4.4.1, we generated the ground truth in the dataset with GPT-4o and manually corrected, deleted, and modified some answers. However, using ground-truth answers created by

human experts could have enabled a more accurate evaluation of the outputs. If experts had written the gold standard answers, the results would probably be more in line with human judgments. This is something that, with more resources, would be a valuable extension of our research to give even more reliable results.

6.3 Broader Impacts and Considerations

As the development of Artificial Intelligence systems becomes more integrated into society, it is important to critically reflect on their impact. Thus, we discuss some ethical, societal, and environmental considerations regarding our work in this section. Even though our system is not in production and we do not have any users, we discuss possible impacts on potential users.

One of the primary ethical concerns in our work is the potential for bias and misleading answers. Research has shown that Large Language Models have inherent biases that originate from the data that the model is trained on [42]. Even though we cannot address this bias in this work, since we are not training the LLMs ourselves, it is important to highlight that the three evaluated models can produce harmful responses. The implementation of a RAG system not only provides the LLM with domain-specific knowledge, but it also serves to mitigate hallucinations and misleading answers on the specific topic. To further address these misleading answers, we also designed a prompt that instructs the model to explicitly respond that it cannot answer the question if the necessary information is not available in the provided context. A challenge with this approach is that the model might overlook the instruction and still produce hallucinated responses. This issue could be addressed by implementing guardrails for the LLM, such as checking the user query for harmful content before it is sent to the model, as well as validating the generated response before it is presented to the user. This solution would probably be needed if we were to put the RAG pipelines in production.

Another ethical concern is how the LLMs handle private or personal data. Our system does not store any user data, only the publicly available manual that the RAG systems leverage. However, we do not account for what the user queries include, meaning that whatever is inputted to our system is sent to one of the used LLMs. Since we rely on APIs to access both Llama 3.2 and DeepSeek-V3, we cannot guarantee that the data sent to these services will not be stored or used by the providers. A solution to this problem could be to run the models locally. This would remove the data leakage as no data will leave the local instance, but it demands substantial computational power depending on the size of the model.

We believe that our system has a positive societal impact as it makes the environmental

certification manuals more accessible. Rather than requiring users to manually search for answers in complex documents, users can directly ask questions and get clear answers. This reduces the effort needed to engage with the manuals, which may lead to a broader use of the certification tool “Miljöbyggnad”. By clarifying the requirements for different environmental grades, the system can also support more effective and informed environmental practices in building projects. We hope that our system could answer in a way that makes the information more user-friendly than it is in the original text. Furthermore, this improved accessibility could lead to an increase in the number of buildings that achieve environmental certifications, which is important for sustainable construction and development.

Continuing with environmental considerations of the system, the development and deployment of LLMs require large computational resources, which in turn lead to high energy consumption and a significant environmental footprint [51]. Given that our research showed GPT-SW3 did not match the performance of the other two models, the necessity of developing language-specific models like this can be questioned from an environmental perspective. While these models promote linguistic inclusion and AI development, it can be argued that the environmental impact of creating these models may be disproportionate to their actual usage and performance.

7 Conclusions

The research in this thesis aimed to gain insight into how the Nordic Large Language Model GPT-SW3, developed by AI Sweden, performed in a Retrieval-Augmented Generation system. This model was compared against two other LLMs developed by major tech companies, Llama 3.2 and DeepSeek-V3. To further assess the performance of the LLMs, we explored how three different Question-Answer RAG systems perform and are affected by the LLMs. These topics are contextualized in our research questions presented in Section 1.2. To answer these questions, we implemented three different RAG pipelines with varying complexity, all based on data from a Swedish manual regarding environmental certification of buildings. The evaluation of the pipelines was conducted by leveraging LLM-as-a-judge, as well as human evaluation by domain experts.

In response to Research Question 1, the results show that GPT-SW3 cannot compete with Llama 3.2 and DeepSeek-V3. The automated evaluation using LLM-as-a-judge rated the responses from the Nordic model significantly lower than the other two models on all metrics. Observing output from the models shows GPT-SW3's inability to leverage large contexts to answer user questions. In addition, the human evaluation confirms these results by also giving the lowest score to GPT-SW3. The conclusion drawn from these results suggests that GPT-SW3 is not competitive from a commercial perspective, despite being trained on predominantly Swedish data. However, the development of language-specific models is still important to include Swedish in future advancements in Natural Language Processing.

Addressing Research Question 2, our results indicate that a more sophisticated RAG pipeline can improve performance. This was especially true when using GPT-SW3, where there was a large performance increase between the naive and the more advanced pipelines. For Llama 3.2 and DeepSeek-V3, the performance gap between the pipelines diminished. These results, along with the findings related to Research Question 1, suggest that selecting a more capable model is more important than developing complex pipelines for this task.

In conclusion, the results presented in this report highlight the challenges faced by language-specific models. A key issue is the limited training data in smaller languages, which correlates with the performance gap between the models in this report. Additionally, the results emphasize that Llama 3.2 and DeepSeek-V3 can perform regardless of the complexity of the pipeline. As LLM continue to improve, their capabilities may reduce the need for complex RAG pipelines, making it easier to develop domain-specific Question-Answer systems.

References

- [1] “Gpt-sw3,” AI Sweden, 2025, accessed: 2025-04-07. [Online]. Available: <https://huggingface.co/AI-Sweden-Models>
- [2] F. Almeida and G. Xexéo, “Word embeddings: A survey,” 2023. [Online]. Available: <https://arxiv.org/abs/1901.09069>
- [3] “Deepseek ups the open weights ante,” The Batch, 2025, accessed: 2025-04-09. [Online]. Available: <https://www.deeplearning.ai/the-batch/deepseek-v3-redefines-llm-performance-and-cost-efficiency/>
- [4] T. B. Brown *et al.*, “Language models are few-shot learners,” 2020. [Online]. Available: <https://arxiv.org/abs/2005.14165>
- [5] S. Bubeck *et al.*, “Sparks of artificial general intelligence: Early experiments with gpt-4,” 2023. [Online]. Available: <https://arxiv.org/abs/2303.12712>
- [6] M. Caballero, “A brief survey of question answering systems,” *International Journal of Artificial Intelligence & Applications*, vol. 12, pp. 01–07, 09 2021.
- [7] O. Cederlund, S. Alawadi, and F. M. Awaysheh, “Llmrag: An optimized digital support service using llm and retrieval-augmented generation,” in *2024 9th International Conference on Fog and Mobile Edge Computing (FMEC)*, 2024, pp. 54–62.
- [8] T. Chander Chandrasekar, “Advancing transportation modeling with large language models : A case study in the swedish context,” Master’s thesis, KTH, Transport planning, 2024.
- [9] H. Chase, “Langchain,” 2022, accessed: 2025-02-10. [Online]. Available: <https://github.com/langchain-ai/langchain>
- [10] B. Chen, Z. Zhang, N. Langrené, and S. Zhu, “Unleashing the potential of prompt engineering in large language models: a comprehensive review,” 2024. [Online]. Available: <https://arxiv.org/abs/2310.14735>
- [11] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, “M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation,” in *Findings of the Association for Computational Linguistics: ACL 2024*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 2318–2335. [Online]. Available: <https://aclanthology.org/2024.findings-acl.137/>

-
- [12] J. Chen, H. Lin, X. Han, and L. Sun, “Benchmarking large language models in retrieval-augmented generation,” 2023. [Online]. Available: <https://arxiv.org/abs/2309.01431>
- [13] N. Chirkova, D. Rau, H. Déjean, T. Formal, S. Clinchant, and V. Nikoulina, “Retrieval-augmented generation in multilingual settings,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.01463>
- [14] A. Darko, A. P. Chan, D.-G. Owusu-Manu, and E. E. Ameyaw, “Drivers for implementing green building technologies: An international survey of experts,” *Journal of Cleaner Production*, vol. 145, pp. 386–394, 2017. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0959652617300501>
- [15] DeepSeek-AI *et al.*, “Deepseek-v3 technical report,” 2025. [Online]. Available: <https://arxiv.org/abs/2412.19437>
- [16] Docker, Inc., “Docker,” 2013. [Online]. Available: <https://www.docker.com>
- [17] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, “The faiss library,” 2025. [Online]. Available: <https://arxiv.org/abs/2401.08281>
- [18] Y. Du, Z. Wang, B. Wang, X. Jin, and Y. Pei, “A novel rag framework with knowledge-enhancement for biomedical question answering,” in *2024 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*, 2024, pp. 3188–3191.
- [19] A. Ekgren, A. Cuba Gyllensten, F. Stollenwerk, J. Öhman, T. Isbister, E. Gogoulou, F. Carlsson, J. Casademont, and M. Sahlgren, “GPT-SW3: An autoregressive language model for the Scandinavian languages,” in *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, N. Calzolari, M.-Y. Kan, V. Hoste, A. Lenci, S. Sakti, and N. Xue, Eds. Torino, Italia: ELRA and ICCL, May 2024, pp. 7886–7900. [Online]. Available: <https://aclanthology.org/2024.lrec-main.695/>
- [20] K. Enevoldsen *et al.*, “Mmteb: Massive multilingual text embedding benchmark,” *arXiv preprint arXiv:2502.13595*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.13595>
- [21] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, “RAGAs: Automated evaluation of retrieval augmented generation,” in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, N. Aletras and O. De Clercq, Eds. St.

- Julians, Malta: Association for Computational Linguistics, Mar. 2024, pp. 150–158. [Online]. Available: <https://aclanthology.org/2024.eacl-demo.16/>
- [22] D. Ferrucci *et al.*, “Building watson: An overview of the deepqa project,” *AI Magazine*, vol. 31, no. 3, pp. 59–79, Jul. 2010. [Online]. Available: <https://ojs.aaai.org/aimagazine/index.php/aimagazine/article/view/2303>
- [23] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” 2024. [Online]. Available: <https://arxiv.org/abs/2312.10997>
- [24] N. Gebremeskel and U. Karunakaran, “An nlp-based method for information retrieval and integration in swedish language,” Master’s thesis, , Microdata Analysis, 2024.
- [25] Google, “Colaboratory,” accessed: 2025-05-08. [Online]. Available: <https://colab.research.google.com>
- [26] A. Grattafiori *et al.*, “The llama 3 herd of models,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [27] B. F. Green, A. K. Wolf, C. Chomsky, and K. Laughery, “Baseball: an automatic question-answerer,” in *Papers Presented at the May 9-11, 1961, Western Joint IRE-AIEE-ACM Computer Conference*, ser. IRE-AIEE-ACM ’61 (Western). New York, NY, USA: Association for Computing Machinery, 1961, p. 219–224. [Online]. Available: <https://doi.org/10.1145/1460690.1460714>
- [28] J. Gu *et al.*, “A survey on llm-as-a-judge,” 2025. [Online]. Available: <https://arxiv.org/abs/2411.15594>
- [29] Y. Han, C. Liu, and P. Wang, “A comprehensive survey on vector database: Storage and retrieval technique, challenge,” 2023. [Online]. Available: <https://arxiv.org/abs/2310.11703>
- [30] O. Holmström, J. Kunz, and M. Kuhlmann, “Bridging the resource gap: Exploring the efficacy of English and multilingual LLMs for Swedish,” in *Proceedings of the Second Workshop on Resources and Representations for Under-Resourced Languages and Domains (RESOURCEFUL-2023)*, N. Ilinykh, F. Morger, D. Dannélls, S. Dobnik, B. Megyesi, and J. Nivre, Eds. Tórshavn, the Faroe Islands: Association for Computational Linguistics, May 2023, pp. 92–110. [Online]. Available: <https://aclanthology.org/2023.resourceful-1.13/>
- [31] Hugging Face, “Hugging Face Inference Providers,” accessed: 2025-05-08. [Online]. Available: <https://huggingface.co/docs/inference-providers/en/index>

-
- [32] M. Joshi, E. Choi, D. S. Weld, and L. Zettlemoyer, “Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension,” 2017. [Online]. Available: <https://arxiv.org/abs/1705.03551>
- [33] S. Krishna, K. Krishna, A. Mohananey, S. Schwarcz, A. Stambler, S. Upadhyay, and M. Faruqui, “Fact, fetch, and reason: A unified evaluation of retrieval-augmented generation,” 2025. [Online]. Available: <https://arxiv.org/abs/2409.12941>
- [34] A. Kshirsagar, “Enhancing rag performance through chunking and text splitting techniques,” *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, vol. 10, pp. 151–158, 09 2024.
- [35] T. Kwiatkowski *et al.*, “Natural questions: A benchmark for question answering research,” *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 452–466, 2019. [Online]. Available: <https://aclanthology.org/Q19-1026/>
- [36] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” 2021. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [37] C.-Y. Lin, “ROUGE: A package for automatic evaluation of summaries,” in *Text Summarization Branches Out*. Barcelona, Spain: Association for Computational Linguistics, Jul. 2004, pp. 74–81. [Online]. Available: <https://aclanthology.org/W04-1013/>
- [38] J. Liu, “LlamaIndex,” 11 2022, accessed: 2025-02-19. [Online]. Available: https://github.com/jerryliu/llama_index
- [39] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” 2023. [Online]. Available: <https://arxiv.org/abs/2307.03172>
- [40] “Llama 3.2,” Meta, 2025, accessed: 2025-04-07. [Online]. Available: <https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct>
- [41] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” 2013. [Online]. Available: <https://arxiv.org/abs/1301.3781>
- [42] R. Navigli, S. Conia, and B. Ross, “Biases in large language models: Origins, inventory, and discussion,” *J. Data and Information Quality*, vol. 15, no. 2, Jun. 2023. [Online]. Available: <https://doi.org/10.1145/3597307>
- [43] R. Nogueira and K. Cho, “Passage re-ranking with bert,” 2020. [Online]. Available: <https://arxiv.org/abs/1901.04085>

-
- [44] OpenAI, “Gpt-4o mini: Advancing cost-efficient intelligence,” 2025, accessed: 2025-05-12. [Online]. Available: <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>
- [45] OpenAI, “Openai api pricing,” 2025, accessed: 2025-05-12. [Online]. Available: <https://openai.com/api/pricing>
- [46] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “Bleu: a method for automatic evaluation of machine translation,” in *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics*, ser. ACL ’02. USA: Association for Computational Linguistics, 2002, p. 311–318. [Online]. Available: <https://doi.org/10.3115/1073083.1073135>
- [47] H. N. Patel, A. Surti, P. Goel, and B. Patel, “A comparative analysis of large language models with retrieval-augmented generation based question answering system,” in *2024 8th International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud) (I-SMAC)*, 2024, pp. 792–798.
- [48] W. Qader, M. M. Ameen, and B. Ahmed, “An overview of bag of words;importance, implementation, applications, and challenges,” pp. 200–204, 06 2019.
- [49] S. Rajapaksha, R. Rani, and E. Karafili, “A rag-based question-answering solution for cyber-attack investigation and attribution,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.06272>
- [50] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “Squad: 100,000+ questions for machine comprehension of text,” 2016. [Online]. Available: <https://arxiv.org/abs/1606.05250>
- [51] M. C. Rillig, M. Ågerstrand, M. Bi, K. A. Gould, and U. Sauerland, “Risks and benefits of large language models for the environment,” *Environmental Science & Technology*, vol. 57, no. 9, pp. 3464–3466, 2023, pMID: 36821477. [Online]. Available: <https://doi.org/10.1021/acs.est.3c01106>
- [52] S. Robertson, “Understanding inverse document frequency: On theoretical arguments for idf,” *Journal of Documentation - J DOC*, vol. 60, pp. 503–520, 10 2004.
- [53] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, “Ares: An automated evaluation framework for retrieval-augmented generation systems,” 2024. [Online]. Available: <https://arxiv.org/abs/2311.09476>
- [54] L. K. Senel, I. Utlu, V. Yucesoy, A. Koc, and T. Cukur, “Semantic structure and interpretability of word embeddings,” *IEEE/ACM Transactions on Audio, Speech,*

- and Language Processing*, vol. 26, no. 10, p. 1769–1779, Oct. 2018. [Online]. Available: <http://dx.doi.org/10.1109/TASLP.2018.2837384>
- [55] S. Setty, H. Thakkar, A. Lee, E. Chung, and N. Vidra, “Improving retrieval for rag based question answering models on financial documents,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.07221>
- [56] R. Shan, “A deep dive into vector stores: Classifying the backbone of retrieval-augmented generation,” in *2024 IEEE International Conference on Big Data (Big-Data)*, 2024, pp. 8831–8833.
- [57] S. Singh and A. Mahmood, “The nlp cookbook: Modern recipes for transformer based deep learning architectures,” *IEEE Access*, vol. 9, pp. 68 675–68 702, 01 2021.
- [58] “Certifiering,” Sweden Green Building Council, accessed: 2025-01-25. [Online]. Available: <https://www.sgbc.se>
- [59] “Om miljöbyggnad,” Swedish Green Building Council, 2025, accessed: 2025-04-11. [Online]. Available: <https://www.sgbc.se/certifiering/miljobyggnad/vad-ar-miljobyggnad/>
- [60] “Unstructured,” Unstructured, 2023, accessed: 2025-02-19. [Online]. Available: <https://github.com/Unstructured-IO/unstructured>
- [61] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” 2023. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [62] M. Velloso and J. Woodward, “Gemini 1.5 pro updates, 1.5 flash debut and 2 new gemma models,” 2024, accessed: 2025-04-09. [Online]. Available: <https://blog.google/technology/developers/gemini-gemma-developer-updates-may-2024/>
- [63] J. Wang *et al.*, “Milvus: A purpose-built vector data management system,” in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 2614–2627.
- [64] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, “Multilingual e5 text embeddings: A technical report,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.05672>
- [65] R. Watanabe, S. Okada, K. Watarai, and T. Mitsunaga, “Cloud secnavigator: Rag approach to bridge gaps and strengthen cloud security practices with ragas assessment,” in *2024 International Conference on Engineering and Emerging Technologies (ICEET)*, 2024, pp. 1–6.

-
- [66] Z. Wei, D. Huang, J. Zhang, C. Song, S. Zhang, J. Zhang, Z. Li, K. Jiang, R. Li, and Q. Duan, “Garag: A general adaptive question-answering system based on rag,” in *Proceedings of the 2024 International Conference on Cloud Computing and Big Data*, ser. ICCBD ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 442–447. [Online]. Available: <https://doi-org.ezproxy.its.uu.se/10.1145/3695080.3695156>
- [67] X. Xie, H. Liu, W. Hou, and H. Huang, “A brief survey of vector databases,” in *2023 9th International Conference on Big Data and Information Analytics (Big-DIA)*, 2023, pp. 364–371.
- [68] H. Yu, A. Gan, K. Zhang, S. Tong, Q. Liu, and Z. Liu, “Evaluation of retrieval-augmented generation: A survey,” 2024. [Online]. Available: <https://arxiv.org/abs/2405.07437>
- [69] G. Zaman, H. Mahdin, K. Hussain, and A. Rahman, “Information extraction from semi and unstructured data sources: A systematic literature review,” *ICIC Express Letters*, vol. 14, no. 6, pp. 593–603, 2020.
- [70] Y. Zhang *et al.*, “Siren’s song in the ai ocean: A survey on hallucination in large language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2309.01219>
- [71] X. Zhao and X. Zhao, “Application of generative artificial intelligence in film image production,” *Computer-Aided Design and Applications*, pp. 15–28, 05 2024.
- [72] K. Zuva and T. Zuva, “Evaluation of information retrieval systems,” *International journal of computer science & information technology*, vol. 4, no. 3, p. 35, 2012.

A Appendix

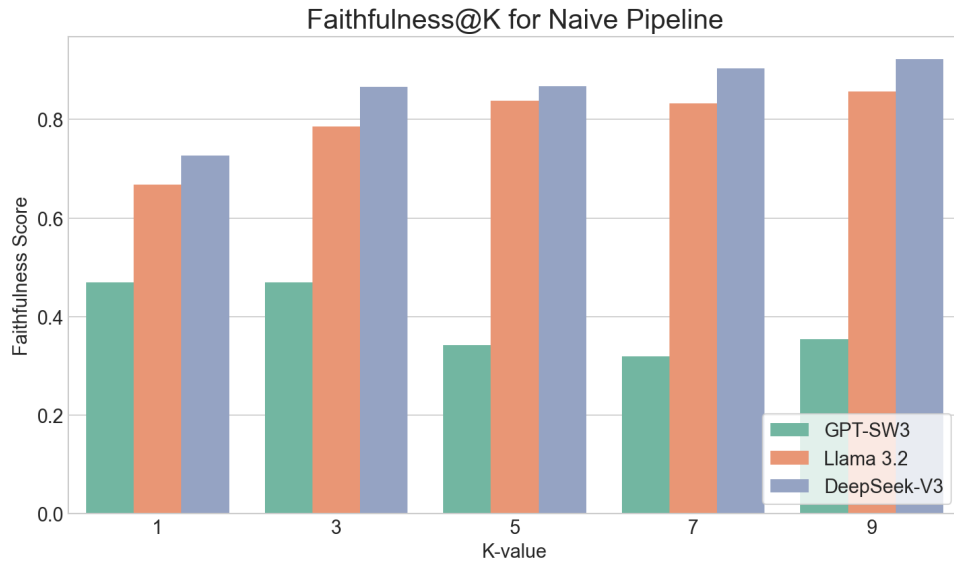


Figure A.1: Comparison of faithfulness scores across three LLMs (GPT-SW3, Llama 3.2, and DeepSeek-V3) in the naive RAG pipeline, evaluated at various K-values (1, 3, 5, 7, and 9). The plot illustrates how the faithfulness scores of each LLM vary with changes in K-value within the naive configuration.

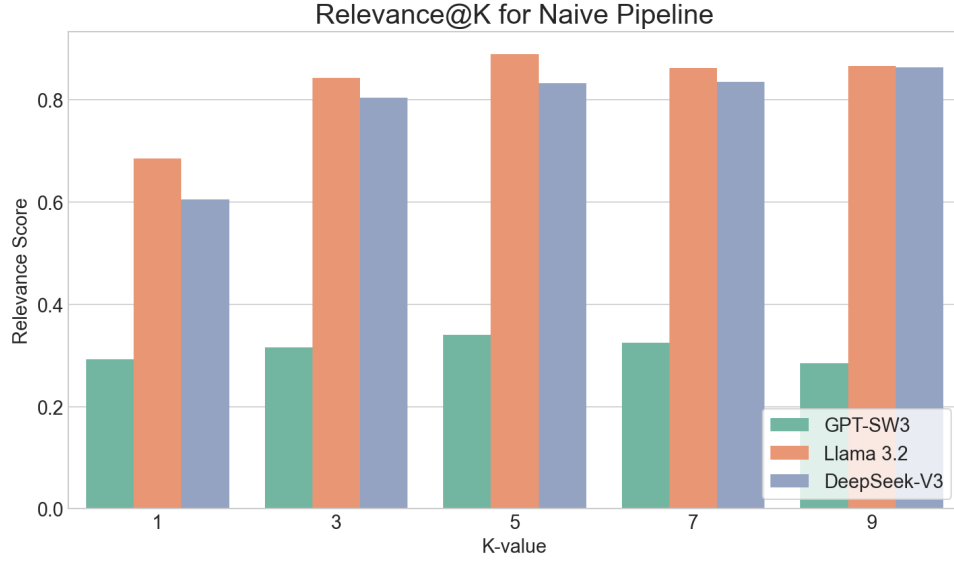


Figure A.2: Comparison of answer relevance scores across three LLMs (GPT-SW3, Llama 3.2, and DeepSeek-V3) in the naive RAG pipeline, evaluated at various K-values (1, 3, 5, 7, and 9). The plot illustrates how the answer relevance scores of each LLM vary with changes in K-value within the naive configuration.

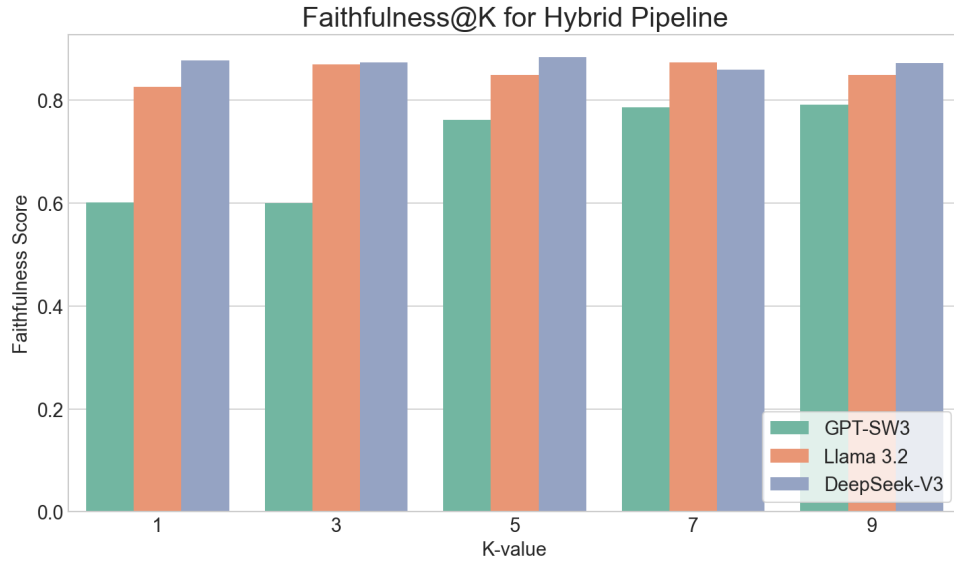


Figure A.3: Comparison of faithfulness scores across three LLMs (GPT-SW3, Llama 3.2, and DeepSeek-V3) in the hybrid RAG pipeline, evaluated at various K-values (1, 3, 5, 7, and 9). The plot illustrates how the faithfulness scores of each LLM vary with changes in K-value within the hybrid configuration.

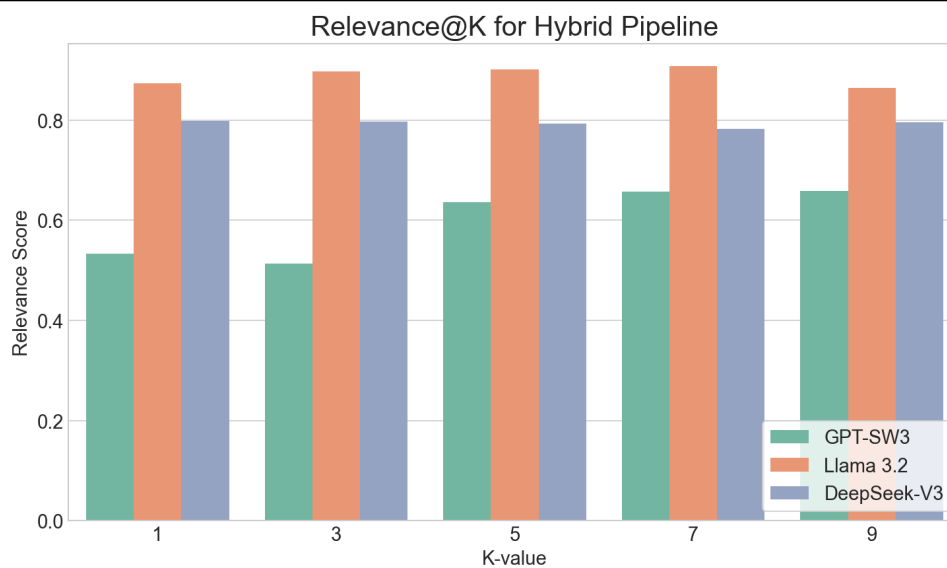


Figure A.4: Comparison of answer relevance scores across three LLMs (GPT-SW3, Llama 3.2, and DeepSeek-V3) in the hybrid RAG pipeline, evaluated at various K-values (1, 3, 5, 7, and 9). The plot illustrates how the answer relevance scores of each LLM vary with changes in K-value within the hybrid configuration.

B Contributions

This thesis was carried out jointly by Markus Lindgren and Lukas Rådmann. All major parts of the project, including research, design, implementation, experimentation, and writing, were completed in close collaboration. We contributed equally to the overall outcome.

More specifically:

- **Markus Lindgren** focused primarily on research regarding embedding models and vector databases, developing the pipelines, and evaluating the retrieval step of the pipelines.
- **Lukas Rådmann** took the lead in the preprocessing and indexing steps, the re-search and selection of large language models, and the automated evaluation with RAGAs.

Although we initially drafted separate sections of the report, we reviewed, revised, and finalized all parts together. As a result, it is not possible to attribute specific sections to one author alone.

Working together as a pair brought significant value to the project. As two people, we were able to explore a wider range of techniques and approaches than would have been possible by one person. This allowed us to test different ideas in parallel and evaluate multiple solutions to each problem. Our different perspectives also led to more thoughtful discussions and better decision-making throughout the process. Besides improving the quality of the work, working together also made the project more enjoyable and productive.